



Bilkent University

EEE 443 - Neural Networks

Second Assignment Report

Fall 2025–2026

Aleyna Yaren Öztürken - 22103420

November 12, 2025

Contents

1	Introduction	1
2	Methodology	1
2.1	The Two Layer Neural Network Architecture	1
2.2	Data Generation	1
2.3	Learning Algorithm: Backpropagation	2
2.3.1	Online Learning	2
2.3.2	Batch Learning	3
3	Hyperparameters and Experimental Setup	5
4	Results and Discussion	5
4.1	Convergence Analysis	5
4.2	The Analysis for the 3000 Epochs for Batch Learning Algorithm	5
4.3	The Analysis for the 3000 Epochs for Online Learning Algorithm	6
4.4	The Analysis for the 7000 Epochs for Batch Learning Algorithm	8
4.5	The Analysis for the 7000 Epochs for Online Learning Algorithm	9
4.6	The Analysis for the 10000 Epochs for Batch Learning Algorithm	11
4.7	The Analysis for the 10000 Epochs for Online Learning Algorithm	12
4.8	The Analysis for the 50000 Epochs for Batch Learning Algorithm	14
4.9	The Analysis for the 50000 Epochs for Online Learning Algorithm	15
5	Overall Comments	17
6	Conclusion	18
7	Appendix	19
A	Full Graphical Results for Online Learning	19
B	Full Graphical Results for Batch Learning	28

1 Introduction

Function approximation is a fundamental problem in machine learning and engineering, where the goal is to find a model that closely matches a target function based on a finite set of data points. Multilayer neural networks are powerful tools for this task due to their ability to learn complex, non-linear relationships. The primary goal is to implement a neural network and a backpropagation algorithm from scratch to train the network. The investigation is based on how different hyperparameters and learning paradigms (online vs. batch) affect the model's ability to converge to a solution with a mean squared error below a target threshold of 10^{-4} .

2 Methodology

The main purpose of this assignment is the implementation of a multilayer neural network to solve a curve fitting problem. The objective is to approximate the following function which,

$$h(x) = 0.25x^2 \sin(x)$$

over the interval $[-\pi, \pi]$. A two-layer neural network was constructed from scratch using Python with the NumPy, Matplotlib and Os libraries. Os library is used for directory and file management. The network's performance was evaluated using two different backpropagation training algorithms being that online (stochastic) learning and batch learning. According to the assignment specifications, experiments must be conducted by varying three key hyperparameters, the number of hidden neurons (n), the learning rate (η), and the data set size (N) across 27 unique combinations. In addition to this hyperparameters, several epoch numbers such as 3000, 7000, 10000, 50000 were also tested to see the impact on convergence. The results show the network's ability to successfully approximate the target function, and a comparative analysis of the learning algorithms and hyperparameter effects on convergence.

2.1 The Two Layer Neural Network Architecture

A two-layer feedforward neural network was designed as specified in the assignment.

- **Input Layer:** The network is designed as having two inputs being that the primary input x and a constant bias input of -1 .
- **Hidden Layer:** This layer contains n neurons, where n was varied as $\{10, 15, 20\}$. The activation function used for each hidden neuron is the hyperbolic tangent ($\tanh$) function, defined as following,

$$f(v) = \tanh(v) = \frac{e^v - e^{-v}}{e^v + e^{-v}}$$

- **Output Layer:** Final layer structured with a single neuron whose activation function is a linear activation function, $f(v) = v$, which directly outputs the network's approximation, $y(x)$.

2.2 Data Generation

According to the assignment specifications, training and testing datasets constructed by sampling the $h(x) = 0.25x^2 \sin(x)$ function which is sampled by the number of data points parameter N , varied as $\{20, 30, 40\}$.

Training Set The training set consists of $8N + 1$ points sampled evenly from the interval $[-\pi, \pi]$. The input values $x(i)$ are given by:

$$x_{\text{train}}(i) = -\pi + \frac{i-1}{4N}\pi, \quad i = 1, 2, \dots, 8N + 1 \quad (1)$$

The corresponding desired outputs $d_{\text{train}}(i)$ are calculated as $d_{\text{train}}(i) = h(x_{\text{train}}(i))$ for all $i = 1, 2, \dots, 8N + 1$.

Test Set The test set consists of $8N$ points, where each point is the midpoint between two consecutive training points. This provides an unseen dataset to evaluate the network's generalization capability.

$$x_{\text{test}}(i) = \frac{x(i) + x(i+1)}{2}, \quad i = 1, 2, \dots, 8N \quad (2)$$

The desired outputs are similarly calculated as $d_{\text{test}}(i) = h(x_{\text{test}}(i))$ for all $i = 1, 2, \dots, 8N$.

2.3 Learning Algorithm: Backpropagation

The backpropagation algorithm was implemented to minimize the error between the network's output and the desired output. The weight update rules are derived from performing gradient descent on the Sum of Squared Errors loss function, which for the entire dataset is:

$$E = \frac{1}{2} \sum_{i=1}^{8N+1} (d(i) - y(i))^2$$

The factor of $\frac{1}{2}$ is a mathematical convenience that simplifies the derivative used for the gradient calculation.

While the weight updates are based on this principle, the overall performance of the network at the end of each epoch was evaluated using the Mean Squared Error (MSE), as specified by the assignment. This metric, denoted as E_{ave} , is calculated as:

$$E_{\text{ave}} = \frac{1}{8N+1} \sum_{i=1}^{8N+1} (d(i) - y(i))^2$$

This E_{ave} value was recorded at the end of each epoch and used as the criterion for convergence. For the convergence comparison, two modes of learning were implemented and compared.

2.3.1 Online Learning

Online learning, also known as stochastic learning, is a training algorithm where the network's weights and biases are updated immediately after the presentation of each individual training sample. The primary advantage of this approach is its potential for faster convergence, as the model parameters are adjusted frequently, allowing the network to start learning from the very first data point. This stochastic nature of the updates, where the gradient is estimated from a single, potentially "noisy" sample, is helpful for optimization because the algorithm can escape from suboptimal "valleys" in the error landscape. A single, noisy update might provide the push needed to climb out of a shallow valley and continue searching for a deeper, more optimal solution that a more stable method would miss. However, this high variance in the gradient estimates is also a key disadvantage. Because it makes the learning process unstable. The calculated error can fluctuate with each update, potentially preventing the model from converging on the optimal solution. Instead, it often ends up just bouncing around the ideal spot. This makes fine-tuning the final convergence more challenging compared to methods that use a more stable, averaged gradient.

Algorithm 1: Online Backpropagation for a Two-Layer Network

- Input:** Training set $\mathcal{S} = \{(x^{(i)}, d^{(i)})\}_{i=1}^M$. Learning rate $\eta > 0$. Number of hidden neurons n . Convergence threshold E_{thresh} . Maximum epochs $MaxEpochs$.
- Output:** Final trained weights $\mathbf{W}^{(1)}, \mathbf{W}^{(2)}$ and biases $\mathbf{b}^{(1)}, b^{(2)}$.
- **Initialize:** weight matrices $\mathbf{W}^{(1)}, \mathbf{W}^{(2)}$ and bias vectors $\mathbf{b}^{(1)}, b^{(2)}$ with small random values.
 - For each epoch from 1 to $MaxEpochs$:
 - For each sample $(x^{(i)}, d^{(i)})$ in $\mathcal{S}$:
 - * **1. Forward Propagation:**
 - Calculate hidden layer net input: $\mathbf{v}^{(1)} \leftarrow \mathbf{W}^{(1)}x^{(i)} - \mathbf{b}^{(1)}$.
 - Calculate hidden layer output: $\mathbf{y}^{(1)} \leftarrow \tanh(\mathbf{v}^{(1)})$.
 - Calculate output layer net input: $v^{(2)} \leftarrow \mathbf{W}^{(2)}\mathbf{y}^{(1)} - b^{(2)}$.
 - Calculate final network output: $y^{(i)} \leftarrow v^{(2)}$ (due to linear activation).
 - * **2. Backward Propagation:**
 - Calculate the error: $e^{(i)} \leftarrow d^{(i)} - y^{(i)}$.
 - Calculate output layer delta: $\delta^{(2)} \leftarrow e^{(i)}$.
 - Calculate hidden layer deltas: $\delta^{(1)} \leftarrow (\mathbf{W}^{(2)})^T \delta^{(2)} \odot (1 - \tanh^2(\mathbf{v}^{(1)}))$.
 - * **3. Update Weights and Biases (Online):**
 - $\mathbf{W}^{(2)} \leftarrow \mathbf{W}^{(2)} + \eta \delta^{(2)}(\mathbf{y}^{(1)})^T$.
 - $b^{(2)} \leftarrow b^{(2)} - \eta \delta^{(2)}$.
 - $\mathbf{W}^{(1)} \leftarrow \mathbf{W}^{(1)} + \eta \delta^{(1)}(x^{(i)})^T$.
 - $\mathbf{b}^{(1)} \leftarrow \mathbf{b}^{(1)} - \eta \delta^{(1)}$.
 - Calculate average error E_{ave} over the entire training set $\mathcal{S}$.
 - If $E_{ave} < E_{thresh}$, break the loop (convergence achieved).
 - Output final weights and biases.
-

2.3.2 Batch Learning

Batch learning, also known as batch gradient descent, is a training algorithm where the network's weights are updated only once per epoch. This update is based on the gradient of the error calculated with respect to the entire training dataset. Unlike online learning, which makes small, frequent updates based on individual samples, the batch method computes an average gradient across all samples. This process provides a more accurate and stable estimate of the "true" gradient of the error surface, leading to a smoother and more direct convergence path toward a local minimum. The primary advantage of this approach is its stability; the weight updates are not influenced by the noise of individual samples, which prevents the sharp oscillations often seen in online learning. However, batch learning has two significant disadvantages. First, it is computationally expensive and slow, as it requires processing every single sample in the dataset before making a single adjustment to the weights. This becomes impractical for very large datasets. Second, because its path is so direct, it has a higher chance of getting permanently stuck in a poor local minimum, whereas the noisy nature of online learning can sometimes help the algorithm escape such minima.

Algorithm 2: Batch Backpropagation for a Two-Layer Network

Input: Training set $\mathcal{S} = \{(x^{(i)}, d^{(i)})\}_{i=1}^M$. Learning rate $\eta > 0$. Number of hidden neurons n . Convergence threshold E_{thresh} . Maximum epochs $MaxEpochs$.

Output: Final trained weights $\mathbf{W}^{(1)}, \mathbf{W}^{(2)}$ and biases $\mathbf{b}^{(1)}, b^{(2)}$.

- **Initialize:** weight matrices $\mathbf{W}^{(1)}, \mathbf{W}^{(2)}$ and bias vectors $\mathbf{b}^{(1)}, b^{(2)}$ with small random values.
 - For each epoch from 1 to $MaxEpochs$:
 - **1. Initialize Gradient Accumulators:** Set the total gradient matrices and vectors to zero.
 - * $\Delta \mathbf{W}^{(1)} \leftarrow \mathbf{0}; \quad \Delta \mathbf{b}^{(1)} \leftarrow \mathbf{0}.$
 - * $\Delta \mathbf{W}^{(2)} \leftarrow \mathbf{0}; \quad \Delta b^{(2)} \leftarrow 0.$
 - **2. Accumulate Gradients over the Batch:**
 - For each sample $(x^{(i)}, d^{(i)})$ in the training set $\mathcal{S}$:
 - * **Forward Propagation:**
 - Calculate hidden layer net input: $\mathbf{v}^{(1)} \leftarrow \mathbf{W}^{(1)}x^{(i)} - \mathbf{b}^{(1)}.$
 - Calculate hidden layer output: $\mathbf{y}^{(1)} \leftarrow \tanh(\mathbf{v}^{(1)}).$
 - Calculate output layer net input: $v^{(2)} \leftarrow \mathbf{W}^{(2)}\mathbf{y}^{(1)} - b^{(2)}.$
 - Calculate final network output: $y^{(i)} \leftarrow v^{(2)}.$
 - * **Backward Propagation:**
 - Calculate the error: $e^{(i)} \leftarrow d^{(i)} - y^{(i)}.$
 - Calculate output layer delta: $\delta^{(2)} \leftarrow e^{(i)}.$
 - Calculate hidden layer deltas: $\delta^{(1)} \leftarrow (\mathbf{W}^{(2)})^T \delta^{(2)} \odot (1 - \tanh^2(\mathbf{v}^{(1)})).$
 - * **Add Sample Gradients to Accumulators:**
 - $\Delta \mathbf{W}^{(2)} \leftarrow \Delta \mathbf{W}^{(2)} + \delta^{(2)}(\mathbf{y}^{(1)})^T.$
 - $\Delta b^{(2)} \leftarrow \Delta b^{(2)} - \delta^{(2)}.$
 - $\Delta \mathbf{W}^{(1)} \leftarrow \Delta \mathbf{W}^{(1)} + \delta^{(1)}(x^{(i)})^T.$
 - $\Delta \mathbf{b}^{(1)} \leftarrow \Delta \mathbf{b}^{(1)} - \delta^{(1)}.$
 - **3. Update Weights and Biases (Once per Epoch):** Update the network parameters using the average gradient over the entire batch.
 - * $\mathbf{W}^{(2)} \leftarrow \mathbf{W}^{(2)} + \eta \cdot \frac{1}{M} \Delta \mathbf{W}^{(2)}.$
 - * $b^{(2)} \leftarrow b^{(2)} + \eta \cdot \frac{1}{M} \Delta b^{(2)}.$
 - * $\mathbf{W}^{(1)} \leftarrow \mathbf{W}^{(1)} + \eta \cdot \frac{1}{M} \Delta \mathbf{W}^{(1)}.$
 - * $\mathbf{b}^{(1)} \leftarrow \mathbf{b}^{(1)} + \eta \cdot \frac{1}{M} \Delta \mathbf{b}^{(1)}.$
 - Calculate average error E_{ave} over the entire training set $\mathcal{S}$.
 - If $E_{ave} < E_{thresh}$, break the loop (convergence achieved).
 - Output final weights and biases.
-

3 Hyperparameters and Experimental Setup

The experiments were conducted by running simulations by two different algorithms as online and batch learning for all 27 combinations of the following hyperparameters:

- **Number of hidden neurons (n):** $\{10, 15, 20\}$
- **Learning rate (η):** $\{0.1, 0.3, 0.5\}$
- **Data density parameter (N):** $\{20, 30, 40\}$

The stopping criterion was set to a several maximum epochs such as 2000, 7000, 10000 and 50000 or until the average training error (E_{ave}) dropped below 10^{-4} . A learning rate decay mechanism was also implemented, reducing η by 10% every 100 epochs to aid convergence. The main purpose of using different number of epochs is to observe the effects of epoch number on convergence behavior and final performance of the network.

4 Results and Discussion

Initially, the implementation was tested for 3000 epochs, with the learning rate decay applied every 50 epochs. However, none of the parameter combinations converged under these settings. Therefore, the learning rate decay mechanism was modified to reduce the learning rate (η) by 10% every 100 epochs. After this adjustment, the error values decreased and approached the threshold of 10^{-4} , though convergence was still not achieved. To further investigate, the epoch limits were increased to 7,000, 10,000, and 50,000 for both online and batch learning algorithms.

4.1 Convergence Analysis

A key observation across all experiments was that the target error of 10^{-4} was not achieved for several hyperparameter combinations within the given epoch limit. This outcome was mentioned by the assignment's hint, which suggests that perfect convergence may not always be possible. The primary reasons for this are the presence of local minima in the error surface and the choice of hyperparameters.

The learning rate η had the most significant impact on performance. One will see results with a smaller learning rate of $\eta = 0.1$ consistently produced the lowest final training and test errors. Higher rates like $\eta = 0.3$ and $\eta = 0.5$ often led to unsteady convergence or caused the algorithm to get stuck in poor local minima with high error, likely due to overshooting the optimal point in the weight space.

4.2 The Analysis for the 3000 Epochs for Batch Learning Algorithm

The following table presents the training and test errors for all 27 hyperparameter combinations after a fixed run of 3000 epochs.

N	n	Learning Rate (η)	Epochs	Train Error	Test Error
20	10	0.100	3000	0.0548047	0.0513088
20	10	0.300	3000	0.0515268	0.0485603
20	10	0.500	3000	0.0511894	0.0482779
20	15	0.100	3000	0.0543279	0.0508069
20	15	0.300	3000	0.0513333	0.0484201
20	15	0.500	3000	0.0494558	0.0467943
20	20	0.100	3000	0.0537558	0.0504462
20	20	0.300	3000	0.0510889	0.0481999
20	20	0.500	3000	0.0496223	0.0469685
30	10	0.100	3000	0.0528114	0.0505104
30	10	0.300	3000	0.0511772	0.0491237
30	10	0.500	3000	0.0499286	0.0480121
30	15	0.100	3000	0.0523569	0.0501217
30	15	0.300	3000	0.0506594	0.0486709
30	15	0.500	3000	0.0484824	0.0466513
30	20	0.100	3000	0.0529489	0.0506710
30	20	0.300	3000	0.0505178	0.0485642
30	20	0.500	3000	0.0490825	0.0472743
40	10	0.100	3000	0.0524567	0.0507450
40	10	0.300	3000	0.0499547	0.0484721
40	10	0.500	3000	0.0499090	0.0484332
40	15	0.100	3000	0.0524070	0.0507110
40	15	0.300	3000	0.0502489	0.0487585
40	15	0.500	3000	0.0485699	0.0472113
40	20	0.100	3000	0.0519739	0.0502959
40	20	0.300	3000	0.0491746	0.0477685
40	20	0.500	3000	0.0486291	0.0472681

Table 1: Summary of Errors for Batch Learning Runs after 3000 Epochs.

An analysis of the batch learning results reveals that the training process is characterized by very slow convergence. After 3000 epochs, the training and test errors across all hyperparameter combinations remain high, clustering around a value of approximately 0.05, which is several orders of magnitude greater than the target threshold of 10^{-4} . Interestingly, the process shows a notable lack of sensitivity to the learning rate, η , as the final errors for any given N and n are remarkably similar across the tested range of 0.1 to 0.5. For instance, with $N = 40$ and $n = 20$, the test errors are all clustered between 0.047 and 0.051. This stability, which is characteristic of batch gradient descent, is accompanied by a lack of recognizable trends related to data density (N) or model capacity (n) at this early stage. The errors are largely grouped together, suggesting that the algorithm’s slow rate of descent is the primary bottleneck, rather than the model’s representative power or the amount of data available. In summary, the batch learning algorithm demonstrates a stable but inefficient training trajectory for this problem, where significant progress would evidently require a substantially greater number of training epochs.

4.3 The Analysis for the 3000 Epochs for Online Learning Algorithm

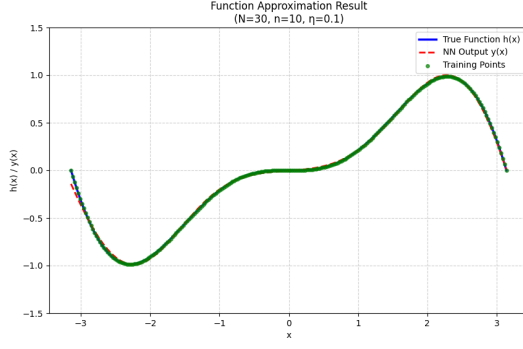
The performance of the online learning algorithm after 3000 epochs is summarized in the following table.

N	n	Learning Rate (η)	Epochs	Train Error	Test Error
20	10	0.100	3000	0.0007866	0.0006570
20	10	0.300	3000	0.0329364	0.0308115
20	10	0.500	3000	0.0338916	0.0321407
20	15	0.100	3000	0.0009527	0.0008067
20	15	0.300	3000	0.0329342	0.0308086
20	15	0.500	3000	0.0341686	0.0324425
20	20	0.100	3000	0.0007283	0.0006138
20	20	0.300	3000	0.0330211	0.0308293
20	20	0.500	3000	0.0012986	0.0011084
30	10	0.100	3000	0.0004224	0.0003787
30	10	0.300	3000	0.0324058	0.0310014
30	10	0.500	3000	0.0045342	0.0040813
30	15	0.100	3000	0.0006630	0.0005891
30	15	0.300	3000	0.0327960	0.0312929
30	15	0.500	3000	0.0334272	0.0322683
30	20	0.100	3000	0.0028585	0.0025873
30	20	0.300	3000	0.0323892	0.0309900
30	20	0.500	3000	0.0342633	0.0331547
40	10	0.100	3000	0.0009667	0.0008866
40	10	0.300	3000	0.0150189	0.0140801
40	10	0.500	3000	0.0013866	0.0012662
40	15	0.100	3000	0.0015274	0.0014022
40	15	0.300	3000	0.0130944	0.0122565
40	15	0.500	3000	0.0333456	0.0324831
40	20	0.100	3000	0.0026774	0.0024876
40	20	0.300	3000	0.0323825	0.0313895
40	20	0.500	3000	0.0275266	0.0262211

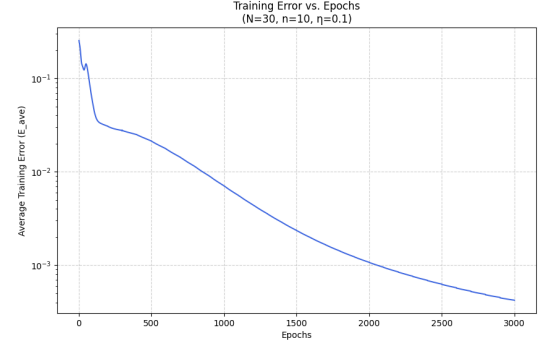
Table 2: Summary of Errors for Online Learning Runs after 3000 Epochs.

The results demonstrate a different behavior compared to the batch method. The most critical observation is the profound sensitivity of the algorithm to the learning rate, η . For the lowest learning rate of $\eta = 0.1$, the network becomes close to converging, with training errors consistently reaching the order of 10^{-4} and getting very close to the target threshold. In sharp contrast, the higher learning rates of $\eta = 0.3$ and $\eta = 0.5$ generally result in significantly worse performance, with errors that are two orders of magnitude higher around 10^{-2} . This large gap suggests that the frequent, noisy updates of the online method can lead to instability and cause the algorithm to overshoot minima when the step size is too large. Within the successful runs using $\eta = 0.1$, the influence of the data density (N) and model capacity (n) is less pronounced, although all combinations manage to getting closer to the convergence. Overall, the online learning approach is highly effective and efficient for this problem, but its success is critically dependent on the careful selection of a sufficiently small learning rate.

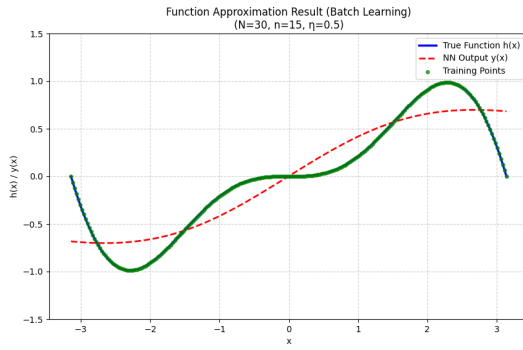
The following figures illustrate the best performing cases from both learning algorithms after 3000 epochs.



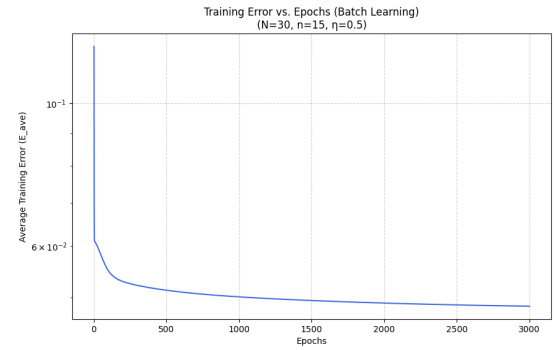
(a) Function approximation using Online Learning ($N=30, n=10, \eta=0.1$).



(b) Training error curve for Online Learning ($N=30, n=10, \eta=0.1$).



(c) Function approximation using Batch Learning ($N=30, n=15, \eta=0.5$).



(d) Training error curve for Batch Learning ($N=30, n=15, \eta=0.5$).

Figure 1: Comparison of function approximation and error convergence for Online and Batch learning algorithms after 3000 epochs. Note the superior fit and lower error achieved by the Online method.

4.4 The Analysis for the 7000 Epochs for Batch Learning Algorithm

To further investigate the convergence behavior of the batch learning algorithm, the training was extended to 7000 epochs. The results for all 27 hyperparameter combinations are presented in the following table.

N	n	Learning Rate (η)	Epochs	Train Error	Test Error
20	10	0.100	7000	0.0543007	0.0507980
20	10	0.300	7000	0.0518307	0.0488249
20	10	0.500	7000	0.0511828	0.0482842
20	15	0.100	7000	0.0536953	0.0503140
20	15	0.300	7000	0.0513617	0.0484368
20	15	0.500	7000	0.0500342	0.0473141
20	20	0.100	7000	0.0537763	0.0504235
20	20	0.300	7000	0.0509329	0.0480786
20	20	0.500	7000	0.0504986	0.0477217
30	10	0.100	7000	0.0530170	0.0507408
30	10	0.300	7000	0.0512034	0.0491593
30	10	0.500	7000	0.0498237	0.0479248
30	15	0.100	7000	0.0531757	0.0508556
30	15	0.300	7000	0.0502966	0.0483511
30	15	0.500	7000	0.0493016	0.0474645
30	20	0.100	7000	0.0515868	0.0494458
30	20	0.300	7000	0.0492414	0.0474160
30	20	0.500	7000	0.0482366	0.0464989
40	10	0.100	7000	0.0521752	0.0504588
40	10	0.300	7000	0.0511135	0.0495440
40	10	0.500	7000	0.0494641	0.0480306
40	15	0.100	7000	0.0525607	0.0508508
40	15	0.300	7000	0.0500351	0.0485514
40	15	0.500	7000	0.0489920	0.0475956
40	20	0.100	7000	0.0527099	0.0509983
40	20	0.300	7000	0.0494299	0.0479960
40	20	0.500	7000	0.0487850	0.0474140

Table 3: Summary of Errors for Batch Learning Runs after 7000 Epochs.

The results after extending the training to 7000 epochs, sustains the initial findings about the batch learning algorithm’s characteristics. The primary observation is the extremely slow rate of convergence. Despite more than doubling the number of training epochs from 3000 to 7000, the reduction in both training and test errors is still remaining high around the 0.05 mark. This confirms that the algorithm is making very slow progress down the error gradient. Furthermore, the behavior remains consistent with previous observations: the performance shows very little sensitivity to the learning rate (η), model capacity (n), or data density (N). The final error values across different hyperparameter settings are still tightly clustered. This persistent pattern of stable but minimal error reduction strongly indicates that the slow convergence is an inherent trait of the batch gradient descent method for this particular problem, rather than a consequence of the hyperparameter choices.

4.5 The Analysis for the 7000 Epochs for Online Learning Algorithm

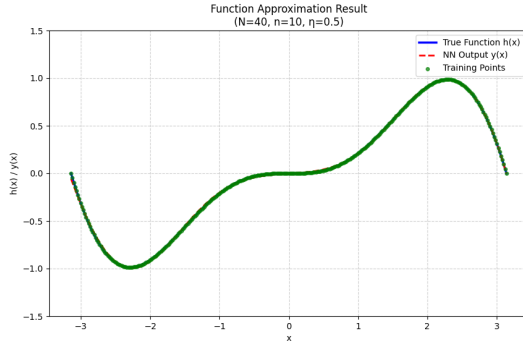
The following results summarize the performance of the online learning algorithm after 7000 epochs.

N	n	Learning Rate (η)	Epochs	Train Error	Test Error
20	10	0.100	7000	0.0004294	0.0003787
20	10	0.300	7000	0.0107562	0.0096898
20	10	0.500	7000	0.0322559	0.0303261
20	15	0.100	7000	0.0010812	0.0009551
20	15	0.300	7000	0.0033819	0.0030312
20	15	0.500	7000	0.0323984	0.0304699
20	20	0.100	7000	0.0007256	0.0006391
20	20	0.300	7000	0.0164595	0.0148959
20	20	0.500	7000	0.0001853	0.0001629
30	10	0.100	7000	0.0002449	0.0002283
30	10	0.300	7000	0.0051560	0.0047788
30	10	0.500	7000	0.0331219	0.0319721
30	15	0.100	7000	0.0003464	0.0003178
30	15	0.300	7000	0.0031746	0.0029412
30	15	0.500	7000	0.0002456	0.0002241
30	20	0.100	7000	0.0020778	0.0019217
30	20	0.300	7000	0.0095650	0.0089150
30	20	0.500	7000	0.0331500	0.0319873
40	10	0.100	7000	0.0004092	0.0003833
40	10	0.300	7000	0.0114588	0.0108674
40	10	0.500	7000	0.0001107	0.0001051
40	15	0.100	7000	0.0003726	0.0003473
40	15	0.300	7000	0.0033743	0.0031833
40	15	0.500	7000	0.0004817	0.0004460
40	20	0.100	7000	0.0004597	0.0004291
40	20	0.300	7000	0.0045771	0.0043226
40	20	0.500	7000	0.0001341	0.0001246

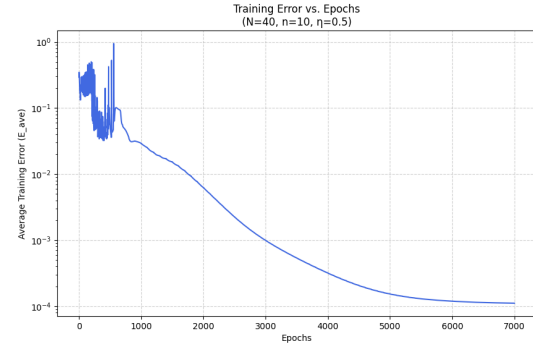
Table 4: Summary of Errors for Online Learning Runs after 7000 Epochs.

The results after 7000 epochs, confirm the high efficiency of the online learning algorithm and reveal a more complex relationship with its hyperparameters. With the extended training time, several hyperparameter combinations have successfully reached very low error values, with some approaching or satisfying the 10^{-4} convergence criterion. The learning rate, η , remains the most critical parameter, but its effect is not monotonic; while the intermediate rate of $\eta = 0.3$ consistently performs poorly, leading to errors in the 10^{-3} to 10^{-2} range, both the lower rate of $\eta = 0.1$ and the higher rate of $\eta = 0.5$ demonstrate strong performance in different scenarios. In fact, some of the best results, such as the run with $N = 40, n = 10, \eta = 0.5$, achieved one of the lowest test errors at 0.0001051. This suggests that the relationship between step size and convergence is non-trivial, and that a larger learning rate can sometimes help escape shallow local minima more effectively over a longer training period. Furthermore, a general trend emerges where higher data density ($N = 40$) and model capacity ($n = 15$ or 20) tend to produce the most robust and lowest-error results, indicating that a more powerful model benefits from more data and sufficient training time.

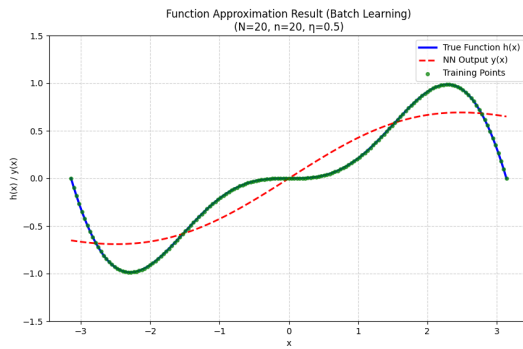
The following figures illustrate the best performing cases from both learning algorithms after 7000 epochs.



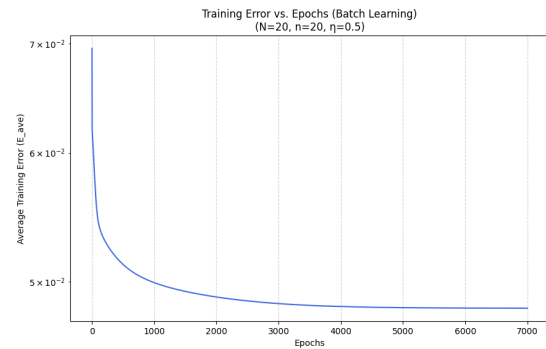
(a) Function approximation using Online Learning (N=40, n=10, $\eta=0.5$).



(b) Training error curve for Online Learning (N=40, n=10, $\eta=0.5$).



(c) Function approximation using Batch Learning (N=20, n=20, $\eta=0.5$).



(d) Training error curve for Batch Learning (N=20, n=20, $\eta=0.5$).

Figure 2: Comparison of a high-performing Online Learning run with a representative Batch Learning run after 7000 epochs. The Online method achieves a perfect fit and an error approaching the 10^{-4} target, while the Batch method's approximation remains poor and its error has plateaued at a high value.

4.6 The Analysis for the 10000 Epochs for Batch Learning Algorithm

The another set of experiments for the batch learning algorithm involved extending the training to a total of 10,000 epochs. The results are presented in the following table.

N	n	Learning Rate (η)	Epochs	Train Error	Test Error
20	10	0.100	10000	0.0539780	0.0506031
20	10	0.300	10000	0.0517949	0.0487857
20	10	0.500	10000	0.0506985	0.0478464
20	15	0.100	10000	0.0537463	0.0502877
20	15	0.300	10000	0.0508639	0.0479956
20	15	0.500	10000	0.0504644	0.0476789
20	20	0.100	10000	0.0531230	0.0498693
20	20	0.300	10000	0.0509139	0.0480645
20	20	0.500	10000	0.0495074	0.0468670
30	10	0.100	10000	0.0526541	0.0503801
30	10	0.300	10000	0.0510232	0.0489858
30	10	0.500	10000	0.0495682	0.0476911
30	15	0.100	10000	0.0519107	0.0497538
30	15	0.300	10000	0.0501928	0.0482653
30	15	0.500	10000	0.0486706	0.0468667
30	20	0.100	10000	0.0527396	0.0505000
30	20	0.300	10000	0.0496209	0.0477543
30	20	0.500	10000	0.0489148	0.0471250
40	10	0.100	10000	0.0516440	0.0499614
40	10	0.300	10000	0.0505510	0.0490316
40	10	0.500	10000	0.0494620	0.0480195
40	15	0.100	10000	0.0518623	0.0501832
40	15	0.300	10000	0.0496298	0.0481782
40	15	0.500	10000	0.0493107	0.0478815
40	20	0.100	10000	0.0519672	0.0503135
40	20	0.300	10000	0.0498626	0.0484078
40	20	0.500	10000	0.0485102	0.0471592

Table 5: Summary of Errors for Batch Learning Runs after 10000 Epochs.

The results after 10,000 epochs conclusively demonstrate that the batch learning algorithm has reached a performance plateau for this problem. Comparing these results with those from the 7,000-epoch mark reveals a negligible decrease in error, confirming that the network’s learning has effectively stalled at a high error level around 0.05. The algorithm continues to exhibit the same characteristics observed previously: a strong insensitivity to the learning rate η , the number of hidden neurons n , and the data density N . The final error values remain tightly clustered across all 27 combinations, indicating that no single hyperparameter setting provides an escape from this high-error local minimum. This strongly suggests that for this specific problem, the nature of the batch gradient descent update rule itself is the limiting factor, preventing the network from converging to a more accurate solution regardless of the training duration or hyperparameter configuration.

4.7 The Analysis for the 10000 Epochs for Online Learning Algorithm

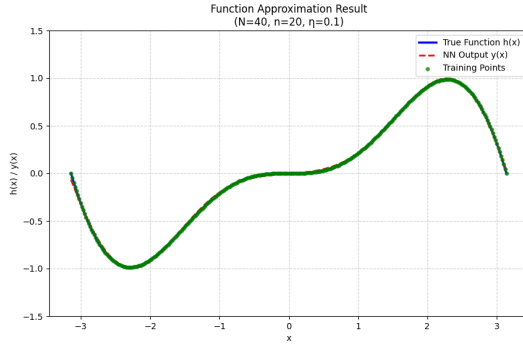
The experiments involved running the online learning algorithm for a total of 10,000 epochs to observe its long-term convergence behavior. The results are summarized in the following table.

N	n	Learning Rate (η)	Epochs	Train Error	Test Error
20	10	0.100	10000	0.0004622	0.0004096
20	10	0.300	10000	0.0024304	0.0021608
20	10	0.500	10000	0.0327583	0.0308934
20	15	0.100	10000	0.0009649	0.0008511
20	15	0.300	10000	0.0194093	0.0178497
20	15	0.500	10000	0.0012721	0.0011226
20	20	0.100	10000	0.0014257	0.0012681
20	20	0.300	10000	0.0291641	0.0273466
20	20	0.500	10000	0.0327227	0.0308511
30	10	0.100	10000	0.0003358	0.0003092
30	10	0.300	10000	0.0063679	0.0059215
30	10	0.500	10000	0.0320817	0.0308229
30	15	0.100	10000	0.0005639	0.0005152
30	15	0.300	10000	0.0034749	0.0032265
30	15	0.500	10000	0.0310217	0.0297543
30	20	0.100	10000	0.0010606	0.0009796
30	20	0.300	10000	0.0105238	0.0098231
30	20	0.500	10000	0.0320061	0.0307370
40	10	0.100	10000	0.0004619	0.0004333
40	10	0.300	10000	0.0218174	0.0209916
40	10	0.500	10000	0.0316992	0.0306784
40	15	0.100	10000	0.0003195	0.0002993
40	15	0.300	10000	0.0019058	0.0017950
40	15	0.500	10000	0.0323791	0.0314961
40	20	0.100	10000	0.0001770	0.0001685
40	20	0.300	10000	0.0303943	0.0294452
40	20	0.500	10000	0.0320989	0.0311806

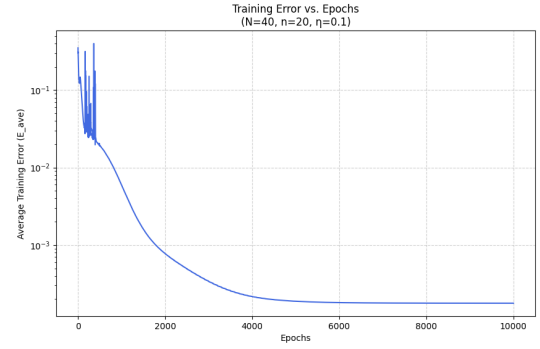
Table 6: Summary of Errors for Online Learning Runs after 10000 Epochs.

The results after 10,000 epochs solidify the conclusion that the online learning algorithm is both highly capable and extremely sensitive to its hyperparameters. The learning rate, η , is unequivocally the most influential factor in determining the final error. A small learning rate of $\eta = 0.1$ consistently guides the network to a low-error solution across all model capacities and data densities, achieving errors on the order of 10^{-4} . In high contrast, larger learning rates of $\eta = 0.3$ and especially $\eta = 0.5$ generally lead to poor convergence, with errors showing at much higher values, often in the 10^{-2} range. This demonstrates the inherent instability of the online update process when the step size is too large. Furthermore, a clear trend emerges showing that the best possible results are obtained when a well-chosen learning rate is combined with high model capacity (n) and high data density (N).

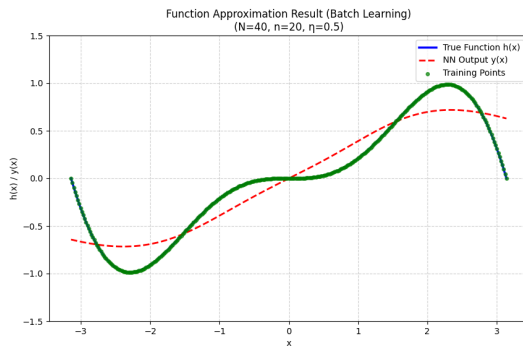
The following figures illustrate the best performing cases from both learning algorithms after 10000 epochs.



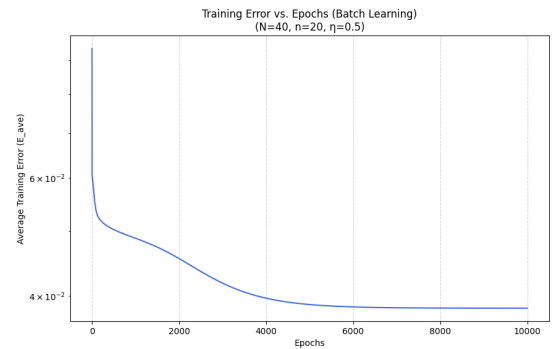
(a) Function approximation using Online Learning ($N=40$, $n=20$, $\eta=0.1$).



(b) Training error curve for Online Learning ($N=40$, $n=20$, $\eta=0.1$).



(c) Function approximation using Batch Learning ($N=40$, $n=20$, $\eta=0.5$).



(d) Training error curve for Batch Learning ($N=40$, $n=20$, $\eta=0.5$).

Figure 3: The comparison of the best Online Learning run with a representative Batch Learning run after 10,000 epochs. The online method achieves a visually perfect fit and its error converges to the target, while the batch method's approximation is poor and its error has long plateaued at a high value.

4.8 The Analysis for the 50000 Epochs for Batch Learning Algorithm

The training was extended to a final run of 50,000 epochs. The results of this extensive training are presented in the following table.

N	n	Learning Rate (η)	Epochs	Train Error	Test Error
20	10	0.100	50000	0.0543261	0.0508163
20	10	0.300	50000	0.0515276	0.0485544
20	10	0.500	50000	0.0505152	0.0476525
20	15	0.100	50000	0.0541155	0.0506990
20	15	0.300	50000	0.0513462	0.0483972
20	15	0.500	50000	0.0498467	0.0471572
20	20	0.100	50000	0.0523169	0.0491486
20	20	0.300	50000	0.0507923	0.0479531
20	20	0.500	50000	0.0498418	0.0471574
30	10	0.100	50000	0.0523102	0.0500982
30	10	0.300	50000	0.0512067	0.0491574
30	10	0.500	50000	0.0500551	0.0481319
30	15	0.100	50000	0.0522260	0.0500796
30	15	0.300	50000	0.0500186	0.0480986
30	15	0.500	50000	0.0493536	0.0475107
30	20	0.100	50000	0.0517673	0.0496108
30	20	0.300	50000	0.0498749	0.0479845
30	20	0.500	50000	0.0486655	0.0469003
40	10	0.100	50000	0.0525620	0.0508580
40	10	0.300	50000	0.0507084	0.0491710
40	10	0.500	50000	0.0494795	0.0480379
40	15	0.100	50000	0.0526167	0.0508888
40	15	0.300	50000	0.0497933	0.0483337
40	15	0.500	50000	0.0486682	0.0472997
40	20	0.100	50000	0.0518526	0.0501925
40	20	0.300	50000	0.0492023	0.0477986
40	20	0.500	50000	0.0484636	0.0471132

Table 7: Summary of Errors for Batch Learning Runs after 50000 Epochs.

The final results offer a definitive conclusion about the performance of the batch learning algorithm on this problem. Comparing these results to those at 10,000 epochs reveals that there has been virtually no improvement in the error metrics. The additional 40,000 epochs of training have failed to produce any meaningful reduction in error, with all values remaining firmly stalled around 0.05. This confirms that the algorithm has converged to a shallow local minimum from which it cannot escape. The training process remains entirely insensitive to the choice of learning rate, model capacity, or data density, as the performance is uniformly poor across all 27 combinations. This demonstrates that for this specific task, the batch learning method is fundamentally unable to reach a low-error solution, and its performance limit is reached relatively early in the training process.

4.9 The Analysis for the 50000 Epochs for Online Learning Algorithm

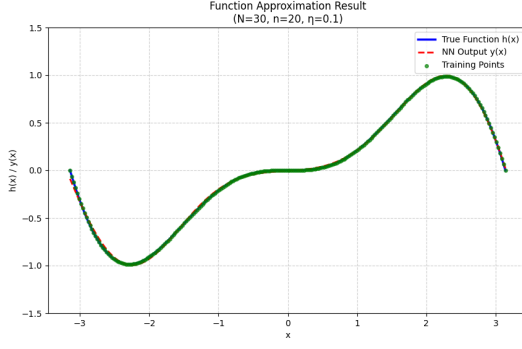
The final training of the online learning algorithm was conducted for 50,000 epochs to assess its convergence capabilities. The comprehensive results are presented in the following table.

N	n	Learning Rate (η)	Epochs	Train Error	Test Error
20	10	0.100	50000	0.0004760	0.0004202
20	10	0.300	50000	0.0158745	0.0143177
20	10	0.500	50000	0.0323937	0.0304875
20	15	0.100	50000	0.0006948	0.0006089
20	15	0.300	50000	0.0066889	0.0060109
20	15	0.500	50000	0.0323233	0.0304154
20	20	0.100	50000	0.0004288	0.0003775
20	20	0.300	50000	0.0038704	0.0034352
20	20	0.500	50000	0.0336985	0.0319513
30	10	0.100	50000	0.0002305	0.0002152
30	10	0.300	50000	0.0026589	0.0024624
30	10	0.500	50000	0.0331185	0.0319581
30	15	0.100	50000	0.0004594	0.0004200
30	15	0.300	50000	0.0030770	0.0028586
30	15	0.500	50000	0.0002562	0.0002339
30	20	0.100	50000	0.0002300	0.0002114
30	20	0.300	50000	0.0055558	0.0051669
30	20	0.500	50000	0.0331209	0.0319618
40	10	0.100	50000	0.0002969	0.0002801
40	10	0.300	50000	0.0110062	0.0104537
40	10	0.500	50000	0.0323284	0.0314283
40	15	0.100	50000	0.0003248	0.0003050
40	15	0.300	50000	0.0199155	0.0191726
40	15	0.500	50000	0.0323041	0.0314015
40	20	0.100	50000	0.0004708	0.0004405
40	20	0.300	50000	0.0025249	0.0023888
40	20	0.500	50000	0.0322458	0.0313382

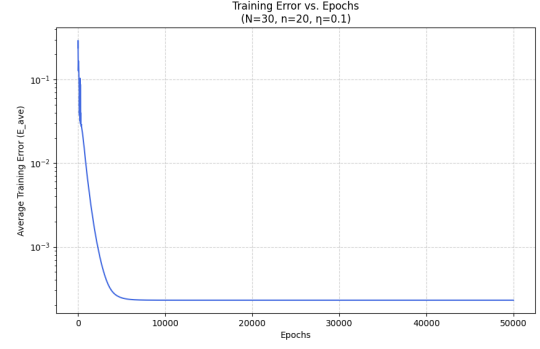
Table 8: Summary of Errors for Online Learning Runs after 50000 Epochs.

The final results after 50,000 epochs provide a clear picture of the online learning algorithm’s capabilities. With sufficient training time, numerous hyperparameter combinations were able to converge to a low-error state, often reaching the order of 10^{-4} . The learning rate, η , remains the most influential parameter, though its effect is complex; the intermediate rate of $\eta = 0.3$ is a clear underperformer, while both the low rate of $\eta = 0.1$ and the high rate of $\eta = 0.5$ can yield excellent results depending on the other parameters. For instance, the combination of $N = 30, n = 15, \eta = 0.5$ produced one of the lowest test errors at 0.0002339, demonstrating that a high learning rate can succeed over a long run. The results also highlight the trend that higher data densities ($N = 30$ and $N = 40$) consistently produce the lowest errors, as seen in the best overall result of 0.0002152 achieved with $N = 30, n = 10, \eta = 0.1$. This indicates that while the learning rate is crucial for enabling convergence, the quality of the final solution is ultimately enhanced by a richer dataset.

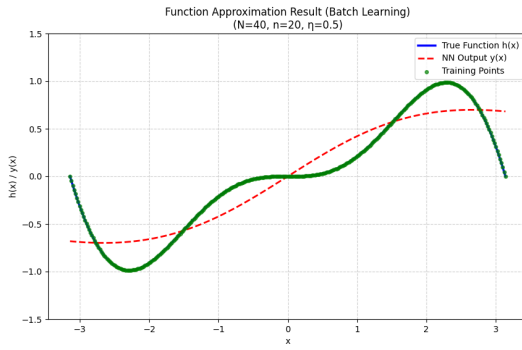
The following figures illustrate the best performing cases from both learning algorithms after 50000 epochs.



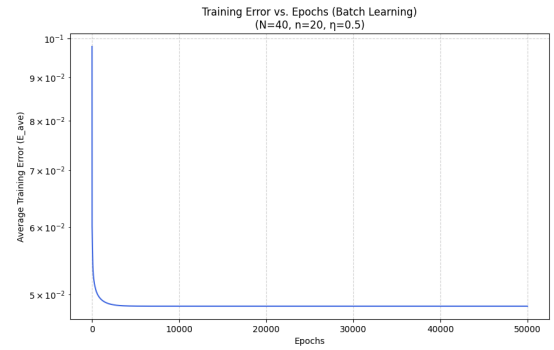
(a) Function approximation using Online Learning ($N=30$, $n=20$, $\eta=0.1$).



(b) Training error curve for Online Learning ($N=30$, $n=20$, $\eta=0.1$).



(c) Function approximation using Batch Learning ($N=40$, $n=20$, $\eta=0.5$).



(d) Training error curve for Batch Learning ($N=40$, $n=20$, $\eta=0.5$).

Figure 4: Final comparison of results after 50,000 epochs. A high-performing Online Learning run (top row) achieves a near-perfect approximation and its error has successfully converged to a very low value. In contrast, the Batch Learning method (bottom row) shows a poor fit and a high, stagnant error.

5 Overall Comments

Across the entire suite of experiments up to 50,000 epochs, a clear and consistent conclusion emerges as the online learning algorithm is way more successful than the batch learning algorithm for this specific function approximation task. The batch method, while stable, proved to be computationally inefficient, as its single, averaged gradient update per epoch led to an extremely slow convergence that ultimately stuck at a high error floor (around 0.05) regardless of the number of epochs or hyperparameter settings. In sharp contrast, the online algorithm's frequent and stochastic updates which is, one for every training sample, allowed it to navigate the complex error surface far more effectively, rapidly descending towards a deep minimum. The optimal performance was consistently achieved using the online algorithm with a low learning rate of $\eta = 0.1$, which provided the necessary stability to prevent the divergence often seen with higher rates. This was best paired with a high data density ($N = 30$ or $N = 40$) and a larger model capacity ($n = 15$ or $n = 20$), which allowed the network to leverage a richer dataset to learn the function's complexities. While the training runs at 50,000 epochs yielded the numerically lowest errors, the principle of diminishing returns was evident; a highly accurate approximation with an error approaching the 10^{-4} target was already achieved by the 10,000 epoch mark, suggesting this represents the most efficient trade-off between computational cost and model accuracy. Since the most successful approach was clearly the online learning algorithm with

50000 epochs, all the related plots are presented in the appendix section.

6 Conclusion

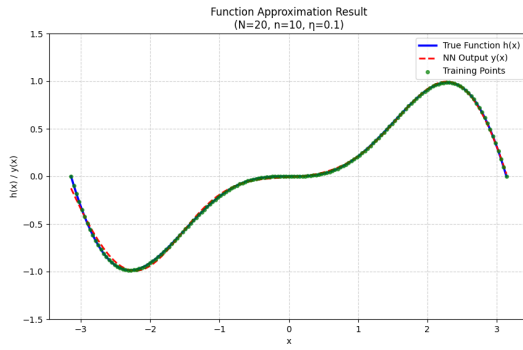
In this assignment, a multilayer neural network was successfully designed and implemented from scratch to approximate the non-linear function $h(x) = 0.25x^2 \sin(x)$. A comprehensive analysis into two learning algorithms demonstrated the major efficiency and effectiveness of the online (stochastic) approach for this task. While the batch algorithm's performance stucked at a high error level regardless of training duration, the online method successfully converged to a solution with an error approaching the target threshold of 10^{-4} . The experiments underscored the critical role of hyperparameter tuning, revealing that a low learning rate ($\eta = 0.1$) was essential for stable convergence in the online mode, while a higher model capacity and data density ($n = 20, N = 40$) contributed to achieving the highest approximation. Consequently, the project not only gave the primary goal of function approximation but also provided practical insights into the fundamental trade-offs between convergence speed and stability inherent in different gradient descent methodologies, confirming the power of neural networks for solving complex regression problems.

7 Appendix

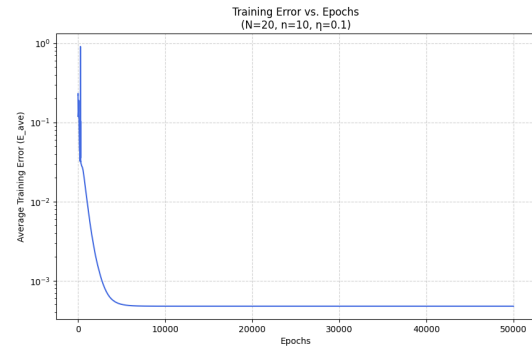
For general analysis, one can check the following drive link to see all the plots generated from the experiments:

[Drive Link for Other Plots](#)

A Full Graphical Results for Online Learning

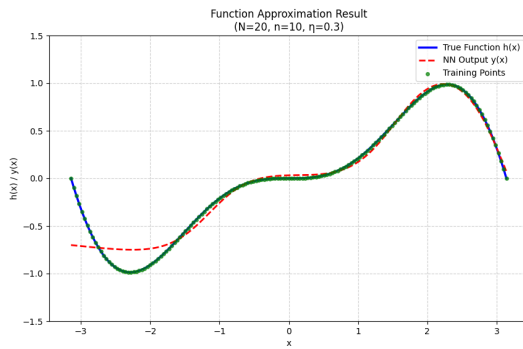


(a) Function Approximation Plot

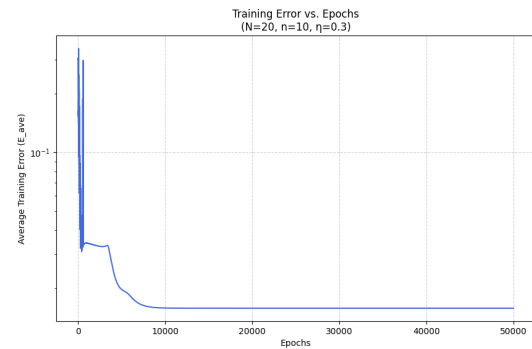


(b) Training Error vs. Epochs

Figure 5: Online Learning Results for $N=20$, $n=10$, $\eta=0.1$.

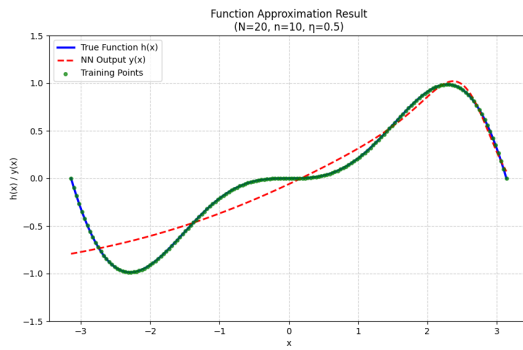


(a) Function Approximation Plot

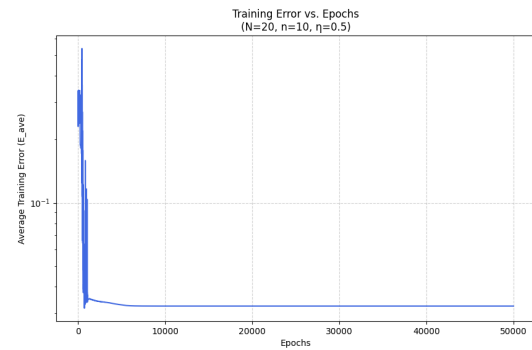


(b) Training Error vs. Epochs

Figure 6: Online Learning Results for $N=20$, $n=10$, $\eta=0.3$.

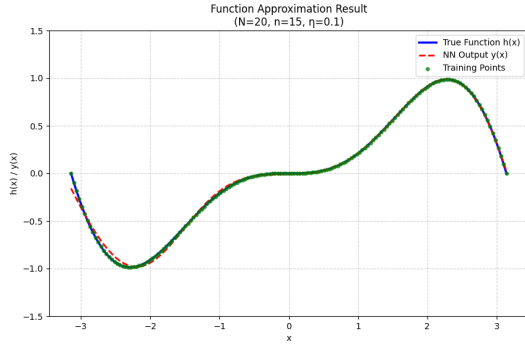


(a) Function Approximation Plot

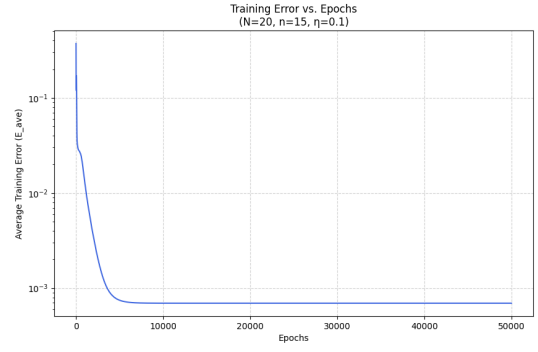


(b) Training Error vs. Epochs

Figure 7: Online Learning Results for $N=20$, $n=10$, $\eta=0.5$.

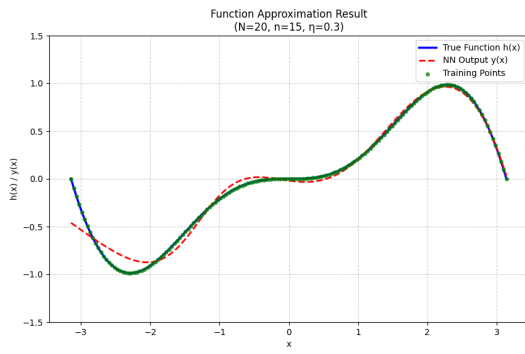


(a) Function Approximation Plot

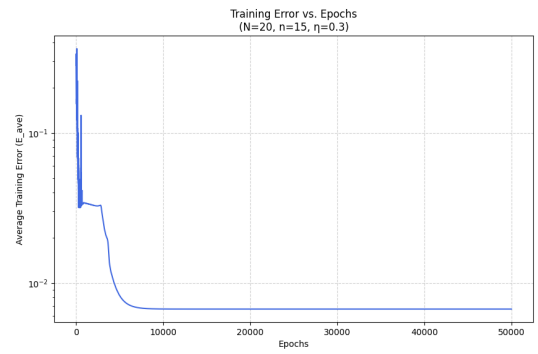


(b) Training Error vs. Epochs

Figure 8: Online Learning Results for $N=20$, $n=15$, $\eta=0.1$.

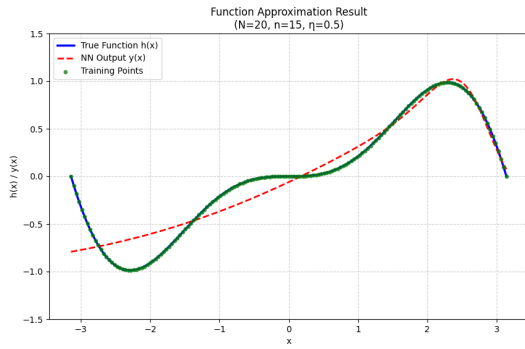


(a) Function Approximation Plot

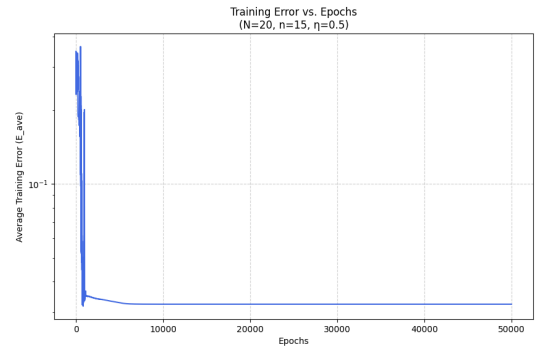


(b) Training Error vs. Epochs

Figure 9: Online Learning Results for $N=20$, $n=15$, $\eta=0.3$.

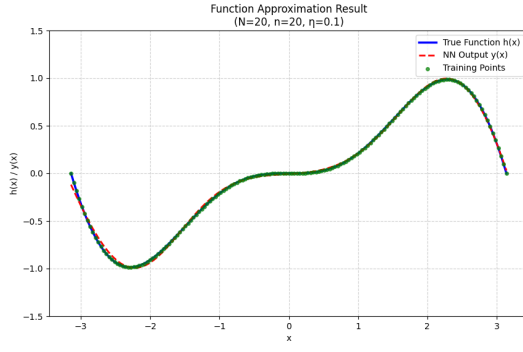


(a) Function Approximation Plot

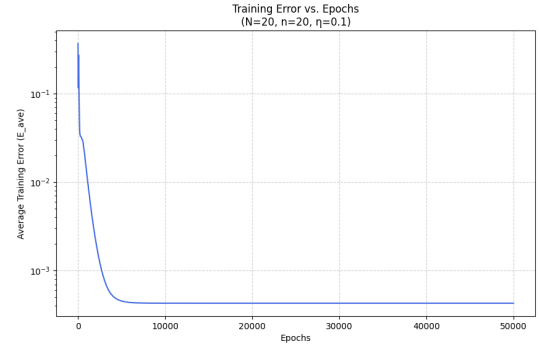


(b) Training Error vs. Epochs

Figure 10: Online Learning Results for $N=20$, $n=15$, $\eta=0.5$.

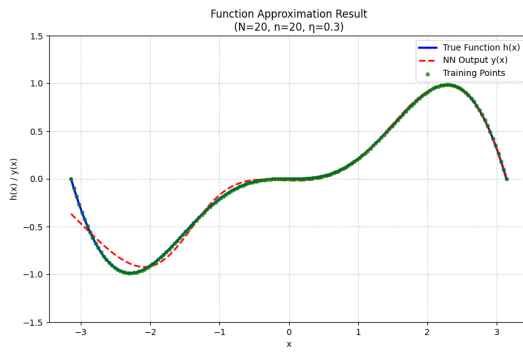


(a) Function Approximation Plot

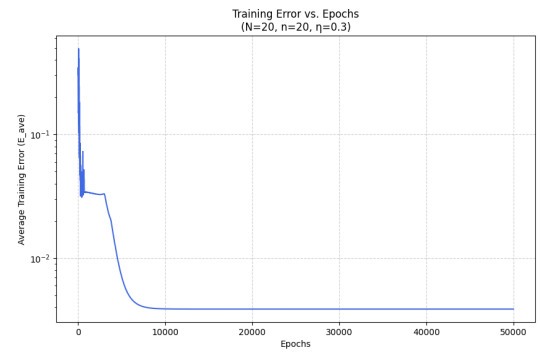


(b) Training Error vs. Epochs

Figure 11: Online Learning Results for $N=20, n=20, \eta=0.1$.

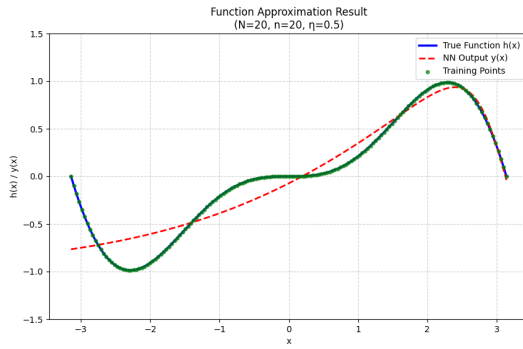


(a) Function Approximation Plot

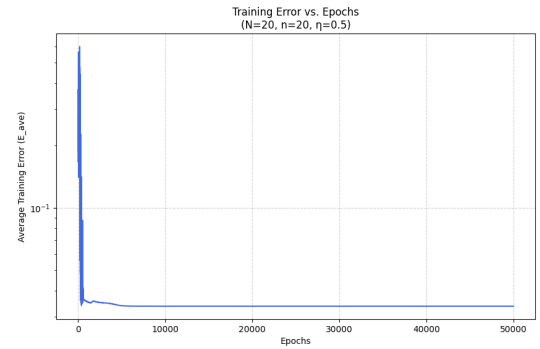


(b) Training Error vs. Epochs

Figure 12: Online Learning Results for $N=20, n=20, \eta=0.3$.

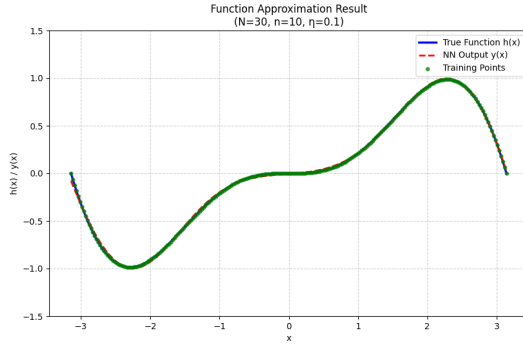


(a) Function Approximation Plot

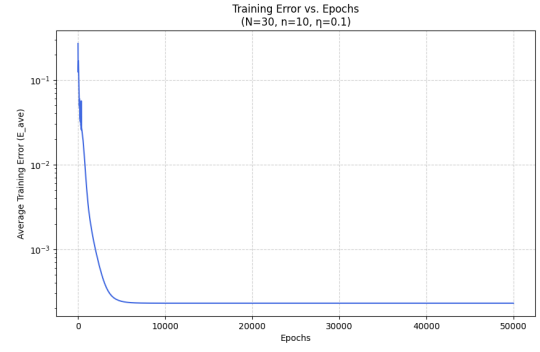


(b) Training Error vs. Epochs

Figure 13: Online Learning Results for $N=20, n=20, \eta=0.5$.

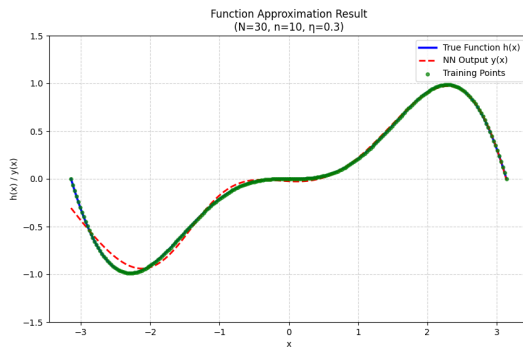


(a) Function Approximation Plot

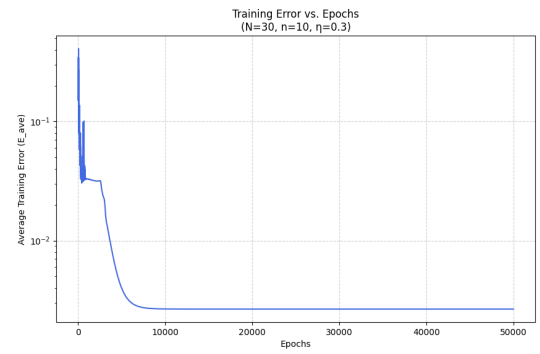


(b) Training Error vs. Epochs

Figure 14: Online Learning Results for $N=30$, $n=10$, $\eta=0.1$.

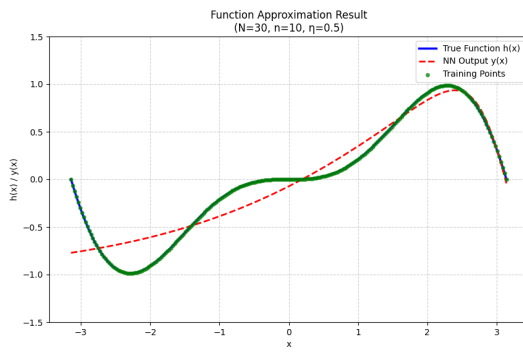


(a) Function Approximation Plot

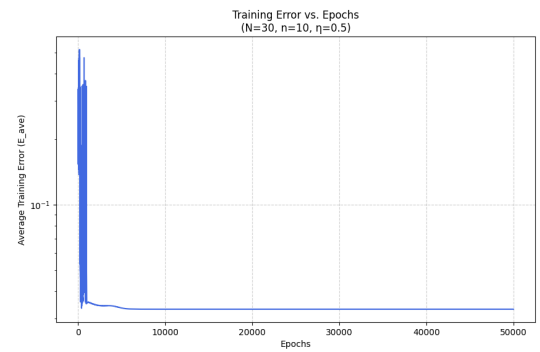


(b) Training Error vs. Epochs

Figure 15: Online Learning Results for $N=30$, $n=10$, $\eta=0.3$.

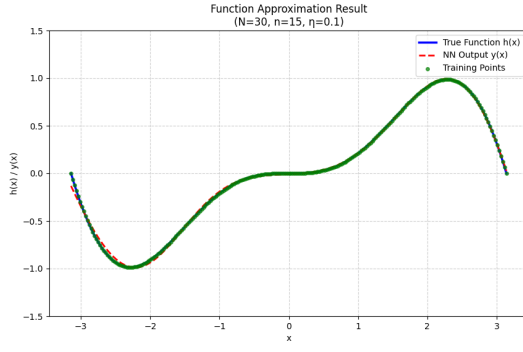


(a) Function Approximation Plot

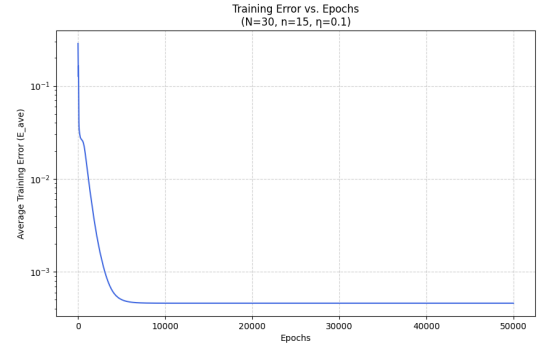


(b) Training Error vs. Epochs

Figure 16: Online Learning Results for $N=30$, $n=10$, $\eta=0.5$.

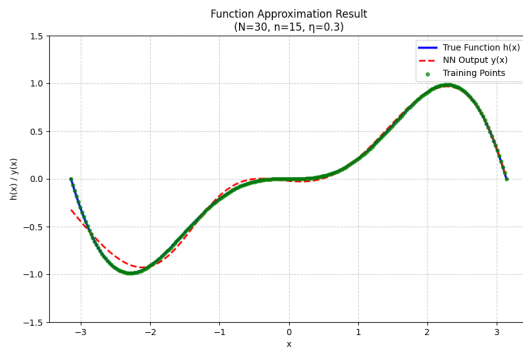


(a) Function Approximation Plot

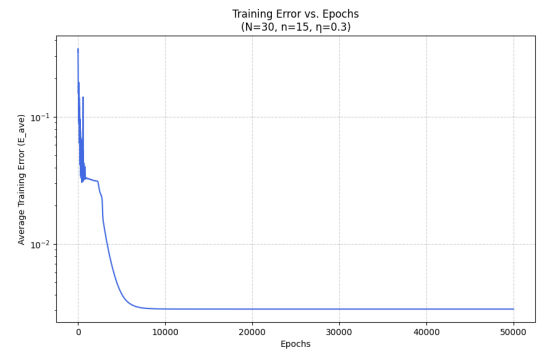


(b) Training Error vs. Epochs

Figure 17: Online Learning Results for $N=30$, $n=15$, $\eta=0.1$.

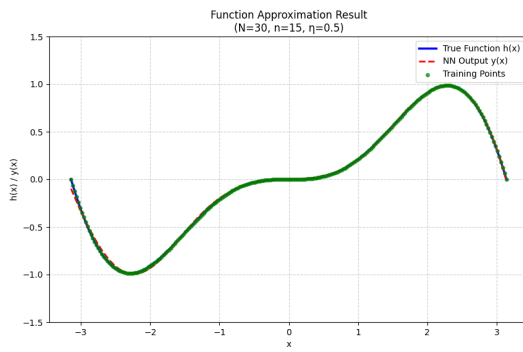


(a) Function Approximation Plot

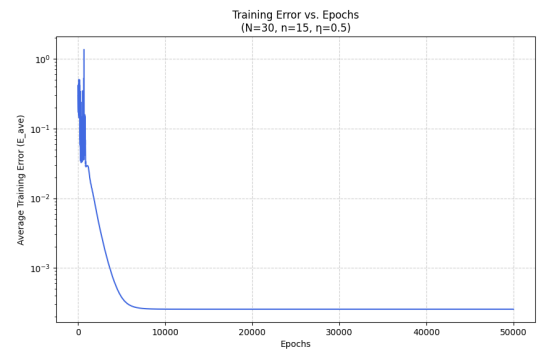


(b) Training Error vs. Epochs

Figure 18: Online Learning Results for $N=30$, $n=15$, $\eta=0.3$.

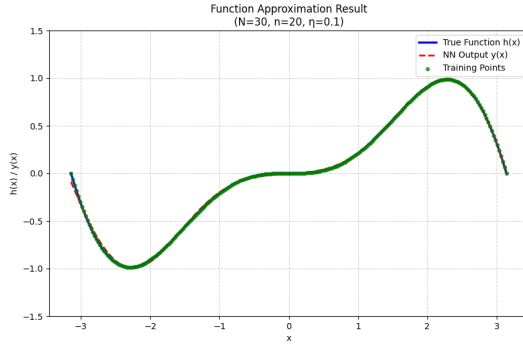


(a) Function Approximation Plot

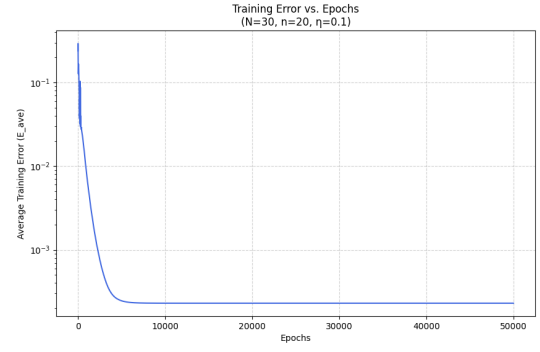


(b) Training Error vs. Epochs

Figure 19: Online Learning Results for $N=30$, $n=15$, $\eta=0.5$.

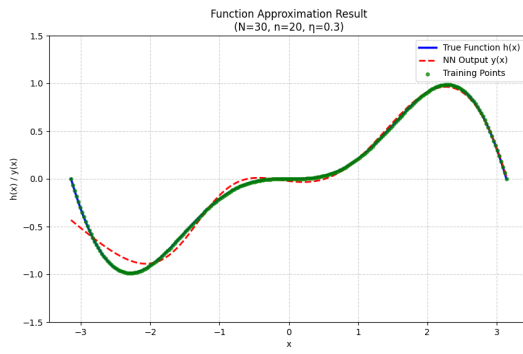


(a) Function Approximation Plot

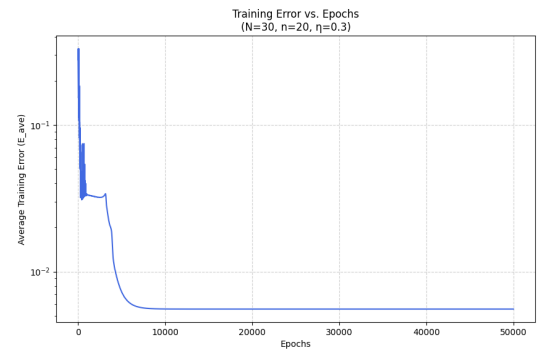


(b) Training Error vs. Epochs

Figure 20: Online Learning Results for $N=30$, $n=20$, $\eta=0.1$.

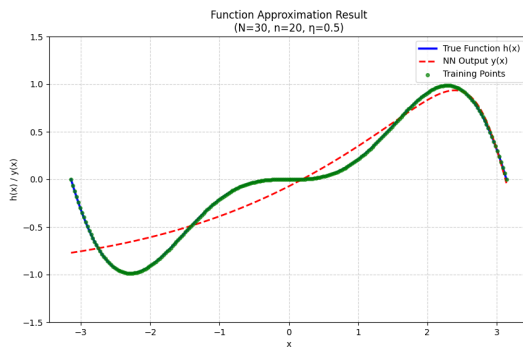


(a) Function Approximation Plot

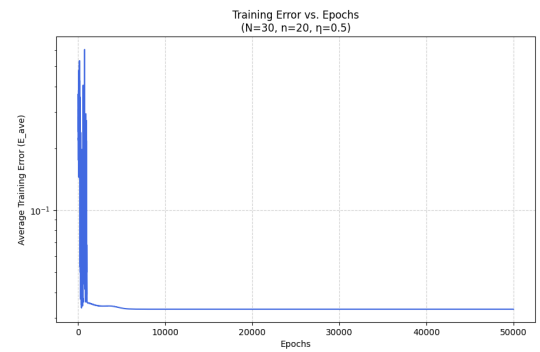


(b) Training Error vs. Epochs

Figure 21: Online Learning Results for $N=30$, $n=20$, $\eta=0.3$.

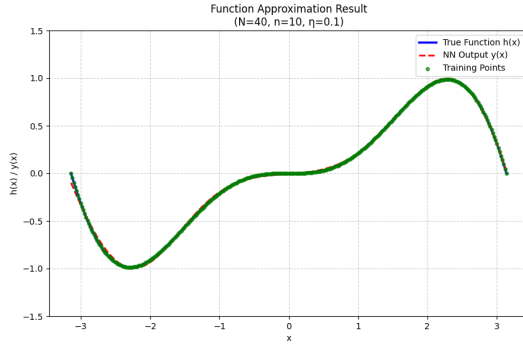


(a) Function Approximation Plot

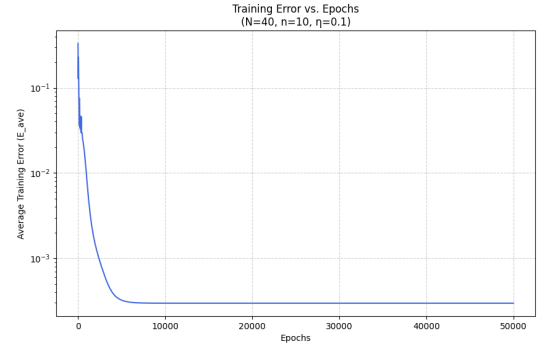


(b) Training Error vs. Epochs

Figure 22: Online Learning Results for $N=30$, $n=20$, $\eta=0.5$.

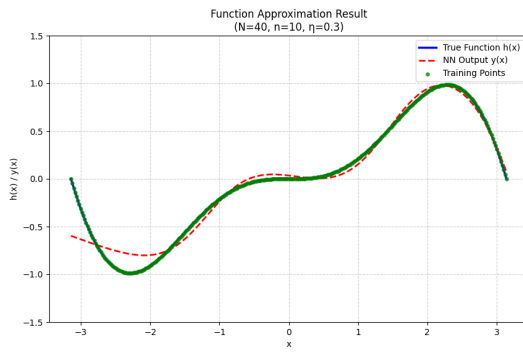


(a) Function Approximation Plot

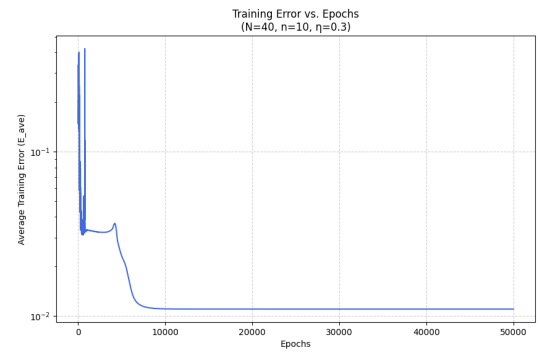


(b) Training Error vs. Epochs

Figure 23: Online Learning Results for $N=40$, $n=10$, $\eta=0.1$.

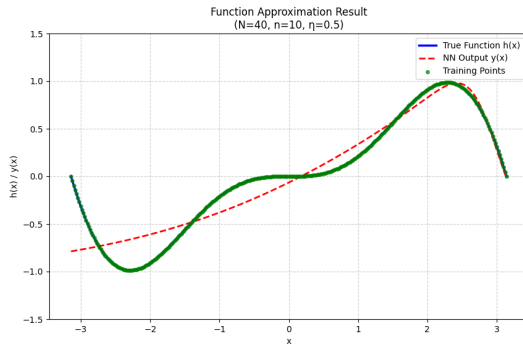


(a) Function Approximation Plot

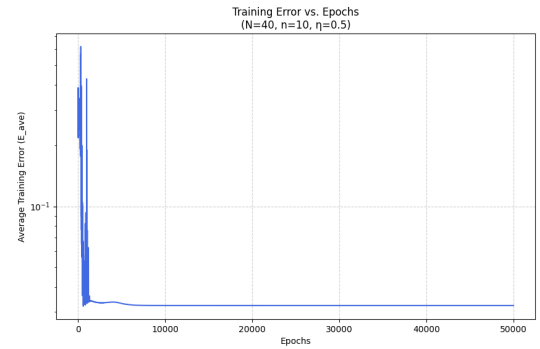


(b) Training Error vs. Epochs

Figure 24: Online Learning Results for $N=40$, $n=10$, $\eta=0.3$.

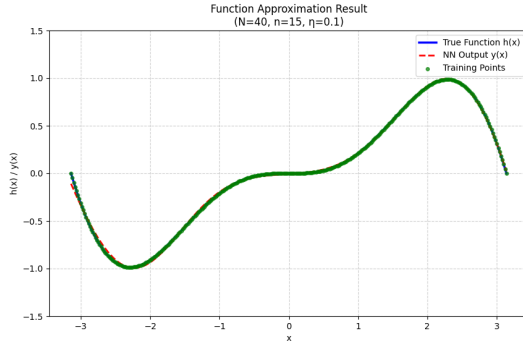


(a) Function Approximation Plot

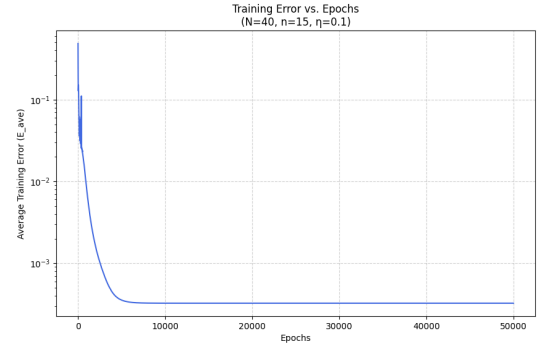


(b) Training Error vs. Epochs

Figure 25: Online Learning Results for $N=40$, $n=10$, $\eta=0.5$.

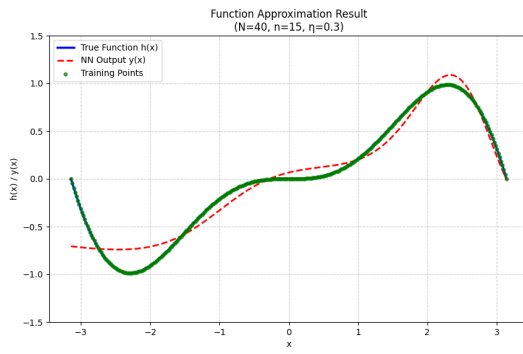


(a) Function Approximation Plot

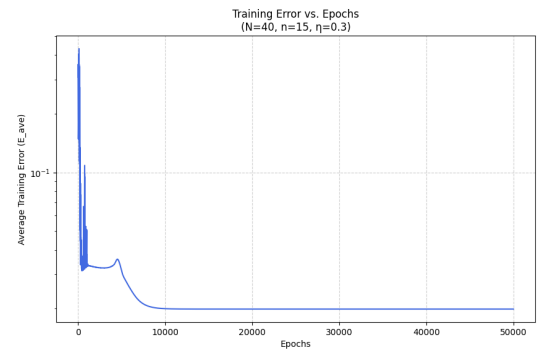


(b) Training Error vs. Epochs

Figure 26: Online Learning Results for $N=40$, $n=15$, $\eta=0.1$.

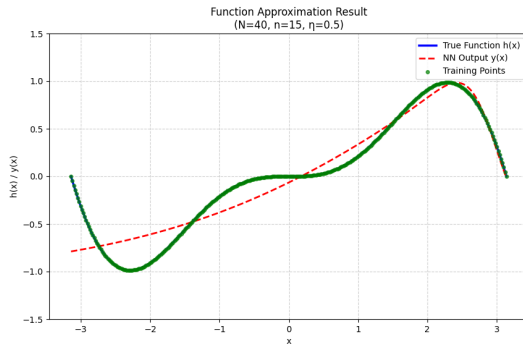


(a) Function Approximation Plot

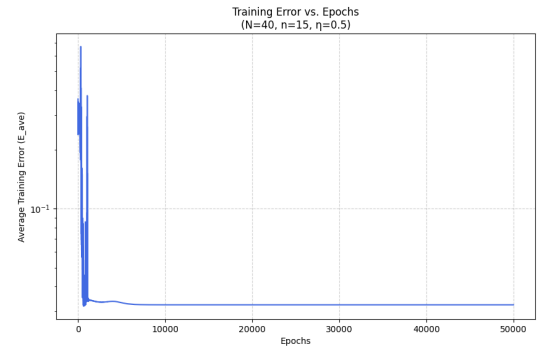


(b) Training Error vs. Epochs

Figure 27: Online Learning Results for $N=40$, $n=15$, $\eta=0.3$.

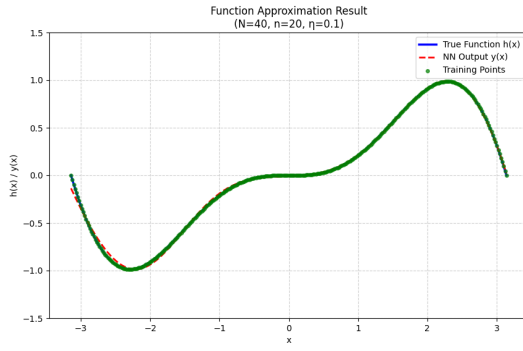


(a) Function Approximation Plot

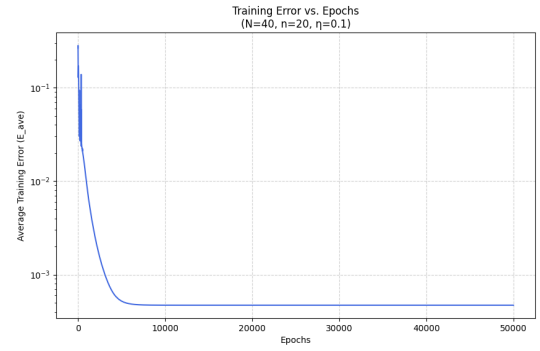


(b) Training Error vs. Epochs

Figure 28: Online Learning Results for $N=40$, $n=15$, $\eta=0.5$.

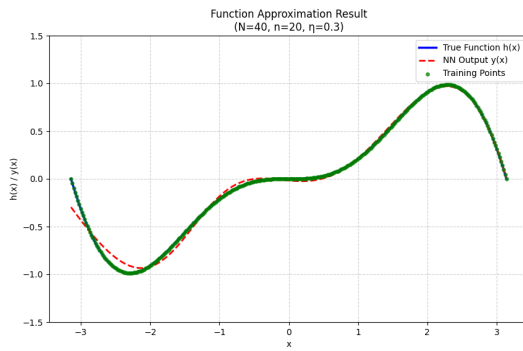


(a) Function Approximation Plot

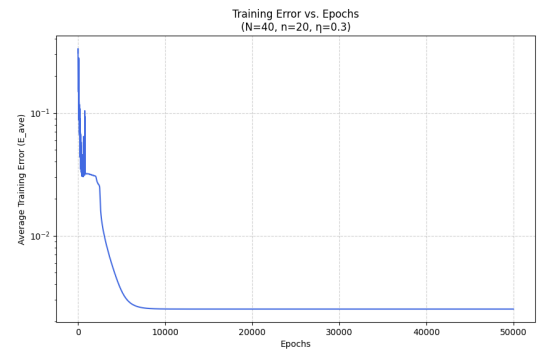


(b) Training Error vs. Epochs

Figure 29: Online Learning Results for $N=40$, $n=20$, $\eta=0.1$.

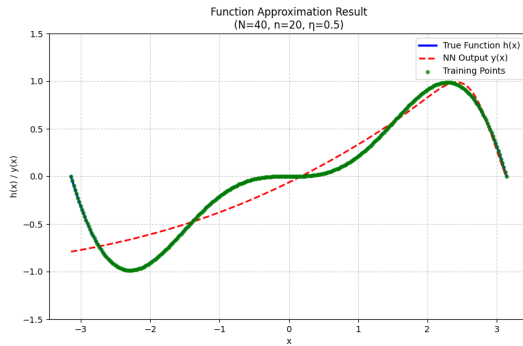


(a) Function Approximation Plot

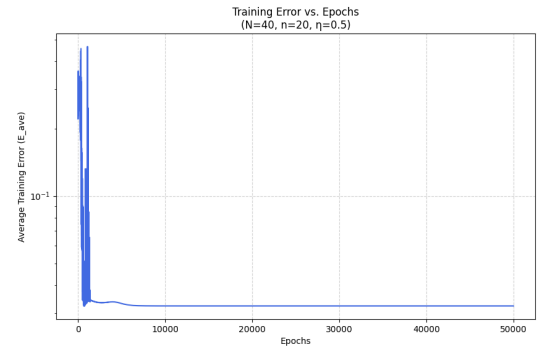


(b) Training Error vs. Epochs

Figure 30: Online Learning Results for $N=40$, $n=20$, $\eta=0.3$.



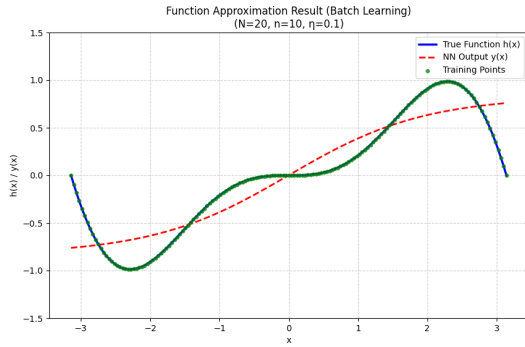
(a) Function Approximation Plot



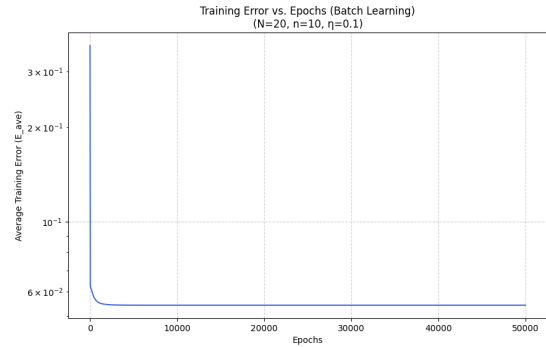
(b) Training Error vs. Epochs

Figure 31: Online Learning Results for $N=40$, $n=20$, $\eta=0.5$.

B Full Graphical Results for Batch Learning

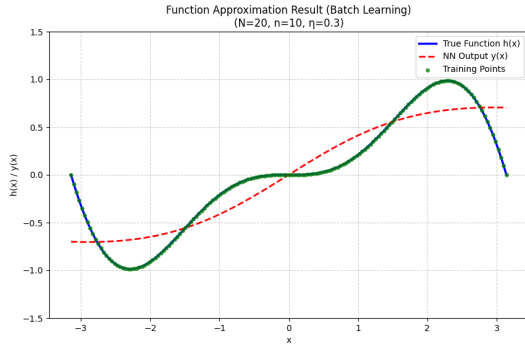


(a) Function Approximation Plot

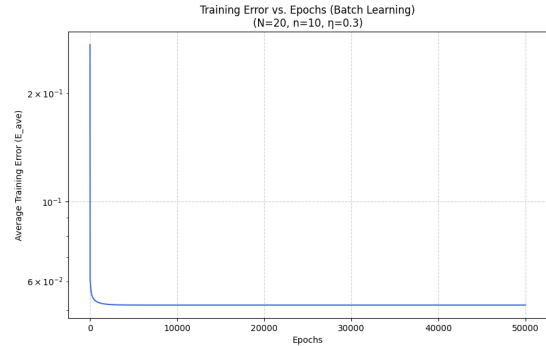


(b) Training Error vs. Epochs

Figure 32: Batch Learning Results for N=20, n=10, $\eta=0.1$.

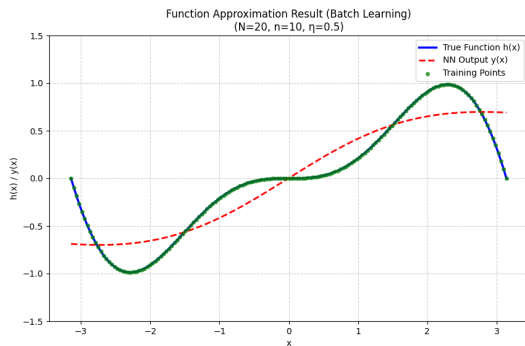


(a) Function Approximation Plot

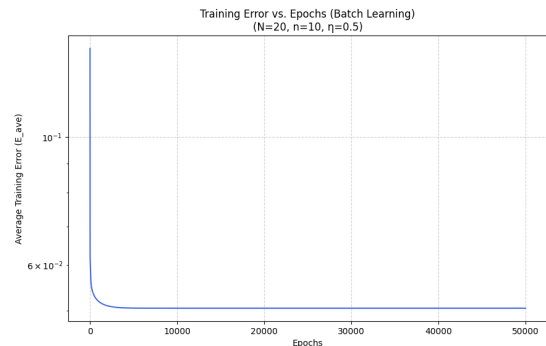


(b) Training Error vs. Epochs

Figure 33: Batch Learning Results for N=20, n=10, $\eta=0.3$.

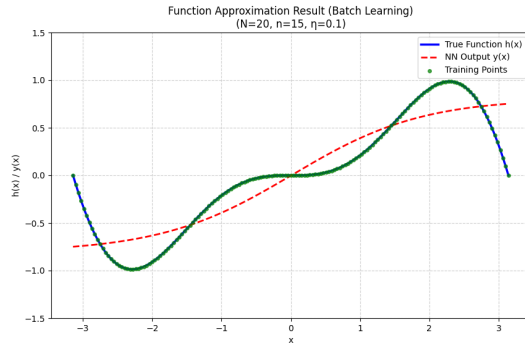


(a) Function Approximation Plot

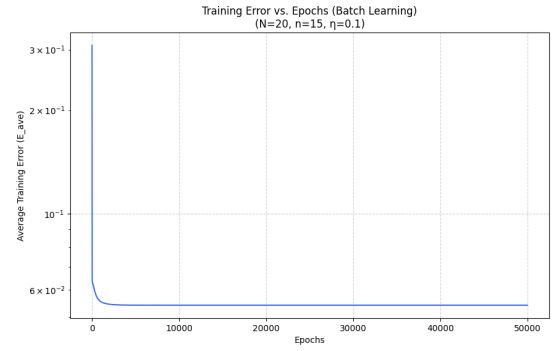


(b) Training Error vs. Epochs

Figure 34: Batch Learning Results for N=20, n=10, $\eta=0.5$.

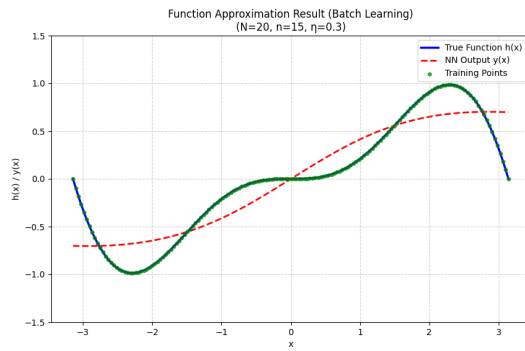


(a) Function Approximation Plot

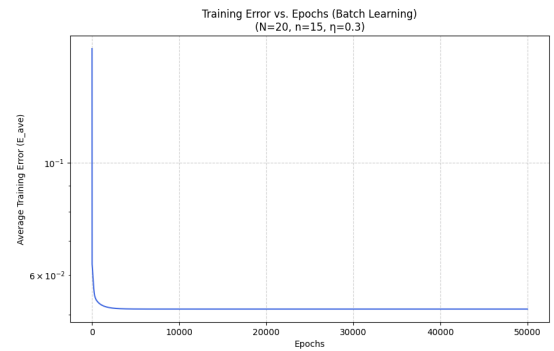


(b) Training Error vs. Epochs

Figure 35: Batch Learning Results for $N=20$, $n=15$, $\eta=0.1$.

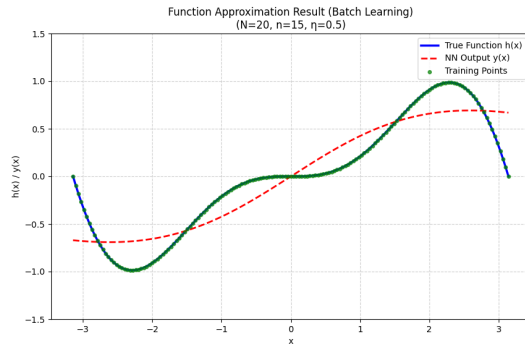


(a) Function Approximation Plot

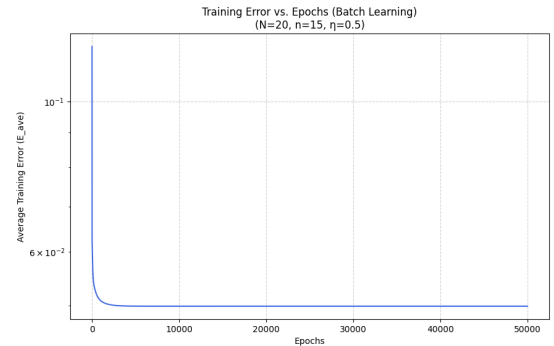


(b) Training Error vs. Epochs

Figure 36: Batch Learning Results for $N=20$, $n=15$, $\eta=0.3$.

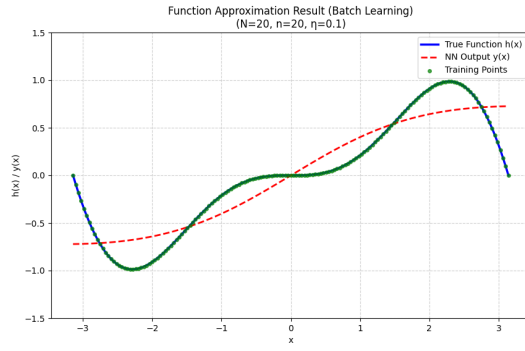


(a) Function Approximation Plot

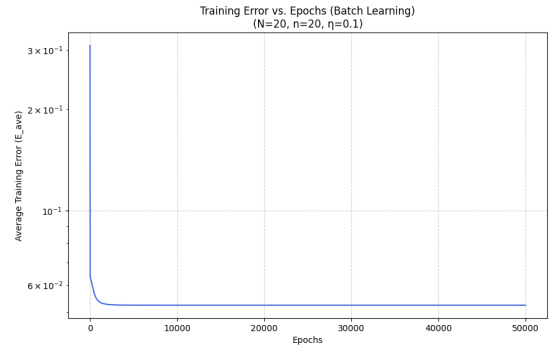


(b) Training Error vs. Epochs

Figure 37: Batch Learning Results for $N=20$, $n=15$, $\eta=0.5$.

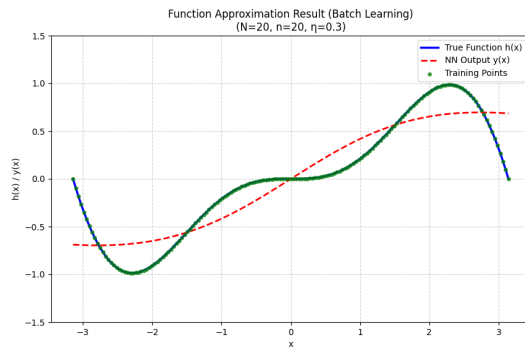


(a) Function Approximation Plot

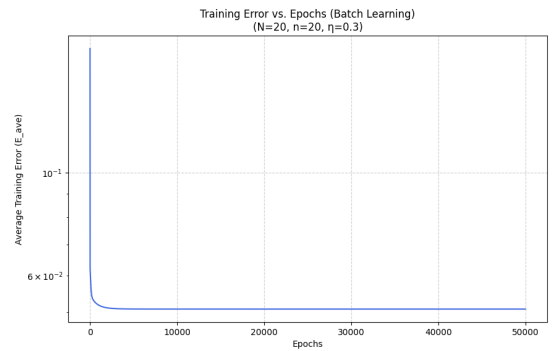


(b) Training Error vs. Epochs

Figure 38: Batch Learning Results for $N=20$, $n=20$, $\eta=0.1$.

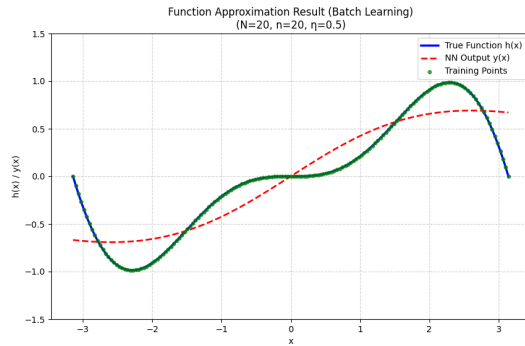


(a) Function Approximation Plot

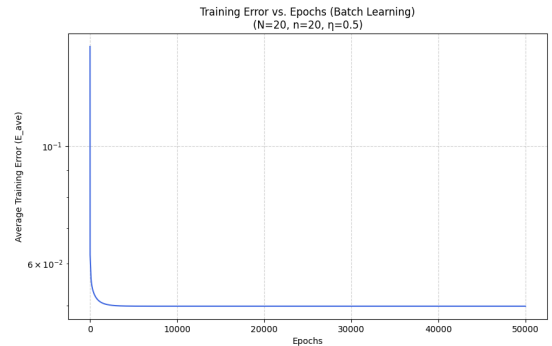


(b) Training Error vs. Epochs

Figure 39: Batch Learning Results for $N=20$, $n=20$, $\eta=0.3$.

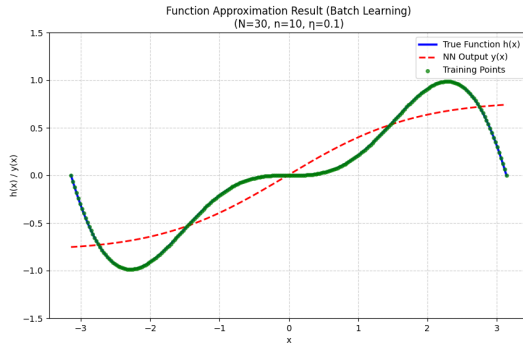


(a) Function Approximation Plot

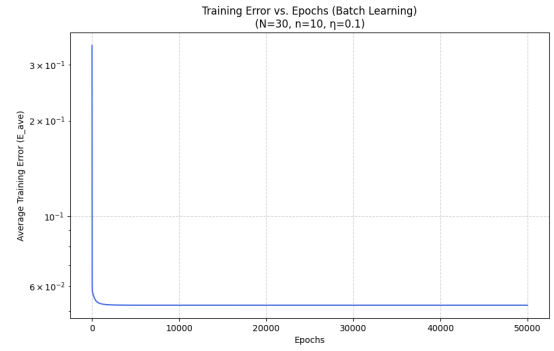


(b) Training Error vs. Epochs

Figure 40: Batch Learning Results for $N=20$, $n=20$, $\eta=0.5$.

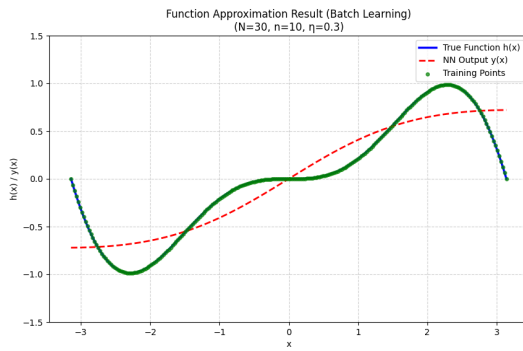


(a) Function Approximation Plot

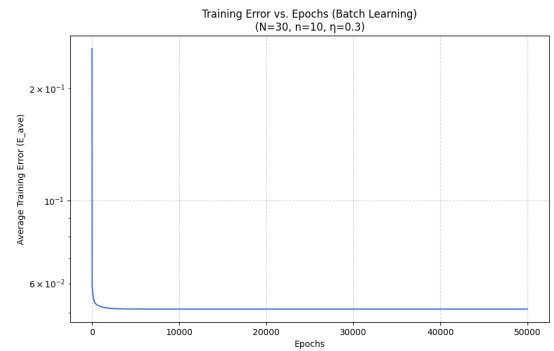


(b) Training Error vs. Epochs

Figure 41: Batch Learning Results for $N=30$, $n=10$, $\eta=0.1$.

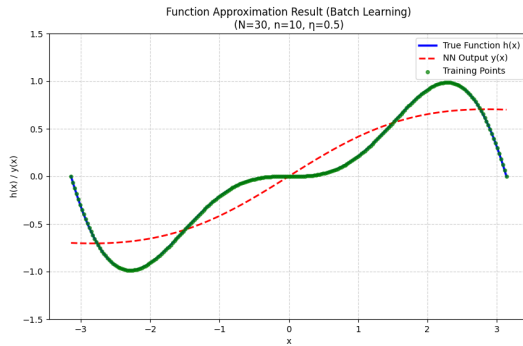


(a) Function Approximation Plot

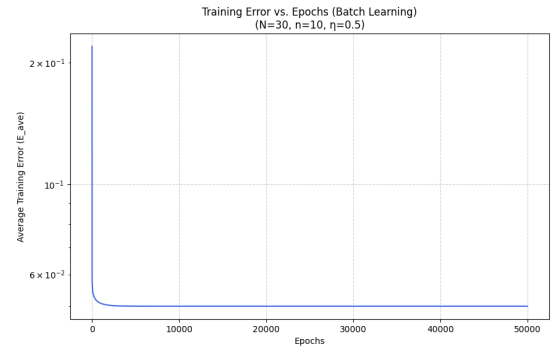


(b) Training Error vs. Epochs

Figure 42: Batch Learning Results for $N=30$, $n=10$, $\eta=0.3$.

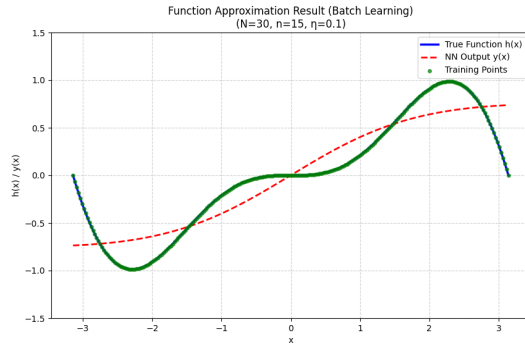


(a) Function Approximation Plot

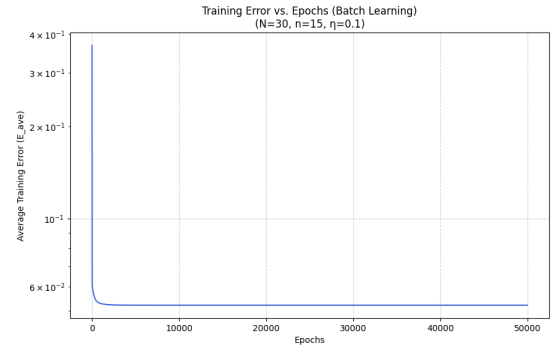


(b) Training Error vs. Epochs

Figure 43: Batch Learning Results for $N=30$, $n=10$, $\eta=0.5$.

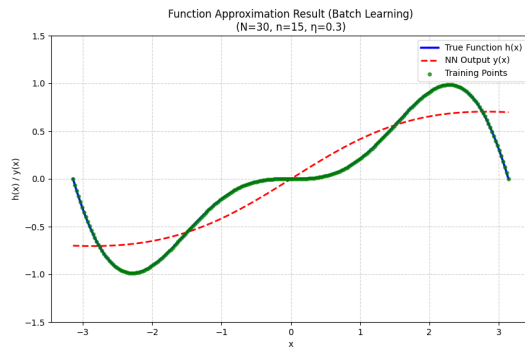


(a) Function Approximation Plot

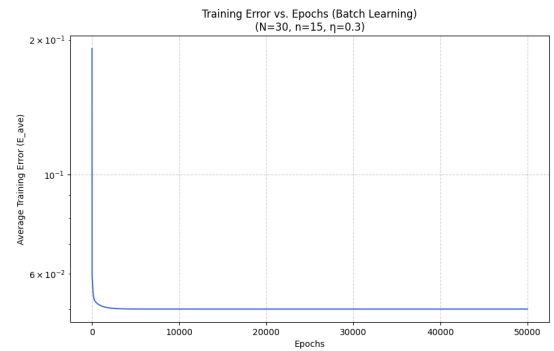


(b) Training Error vs. Epochs

Figure 44: Batch Learning Results for $N=30$, $n=15$, $\eta=0.1$.

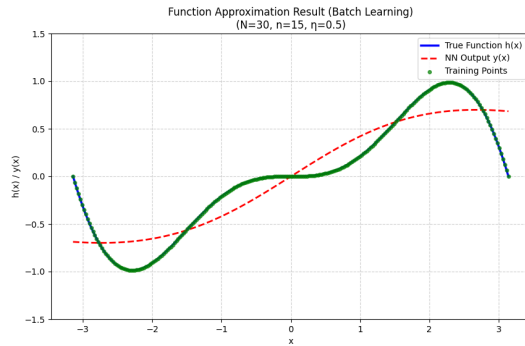


(a) Function Approximation Plot

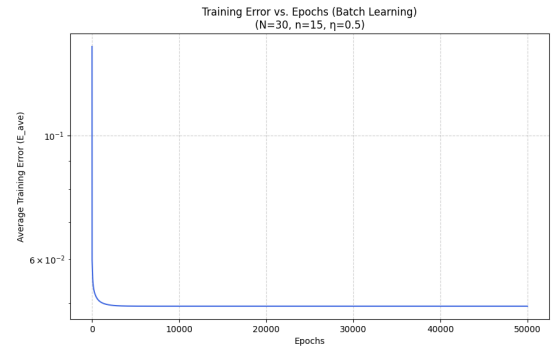


(b) Training Error vs. Epochs

Figure 45: Batch Learning Results for $N=30$, $n=15$, $\eta=0.3$.

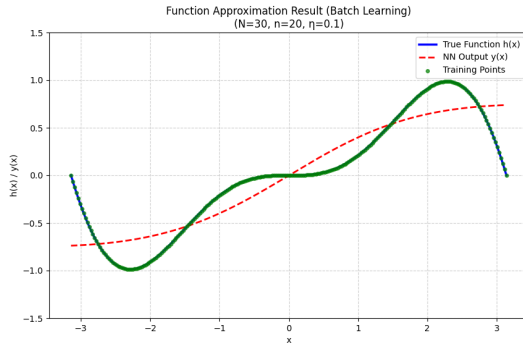


(a) Function Approximation Plot

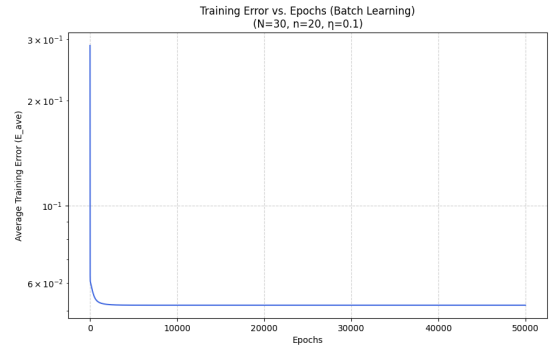


(b) Training Error vs. Epochs

Figure 46: Batch Learning Results for $N=30$, $n=15$, $\eta=0.5$.

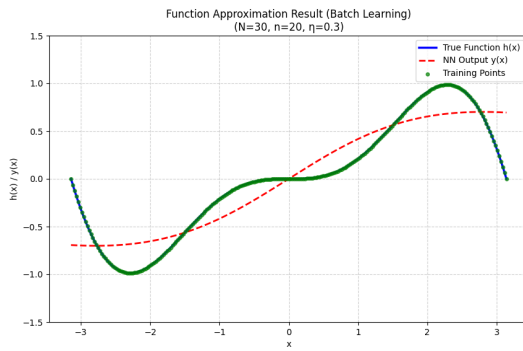


(a) Function Approximation Plot

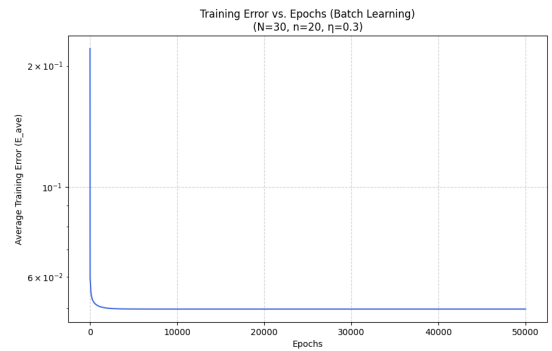


(b) Training Error vs. Epochs

Figure 47: Batch Learning Results for $N=30$, $n=20$, $\eta=0.1$.

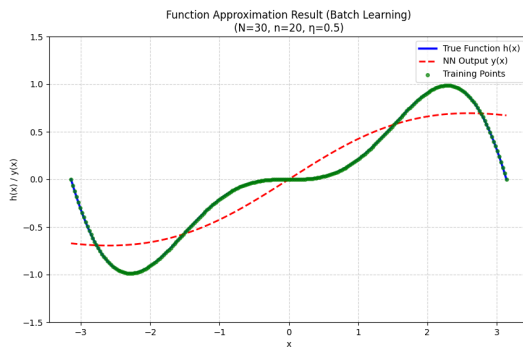


(a) Function Approximation Plot

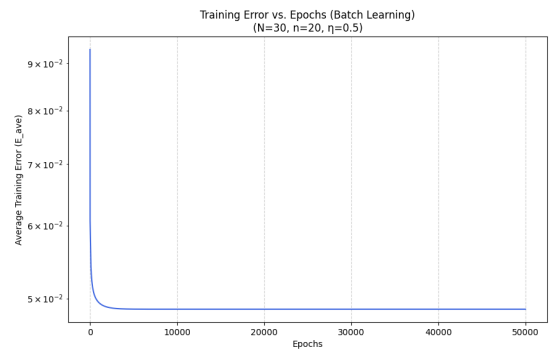


(b) Training Error vs. Epochs

Figure 48: Batch Learning Results for $N=30$, $n=20$, $\eta=0.3$.

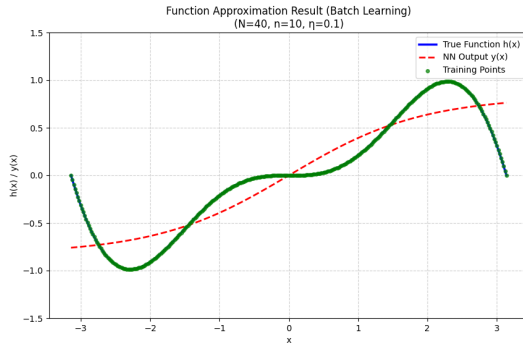


(a) Function Approximation Plot

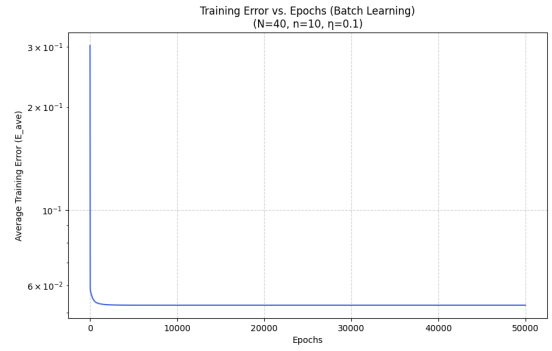


(b) Training Error vs. Epochs

Figure 49: Batch Learning Results for $N=30$, $n=20$, $\eta=0.5$.

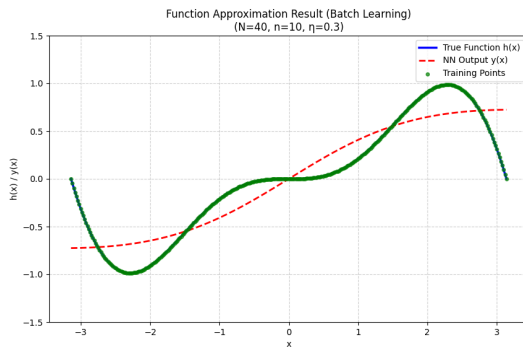


(a) Function Approximation Plot

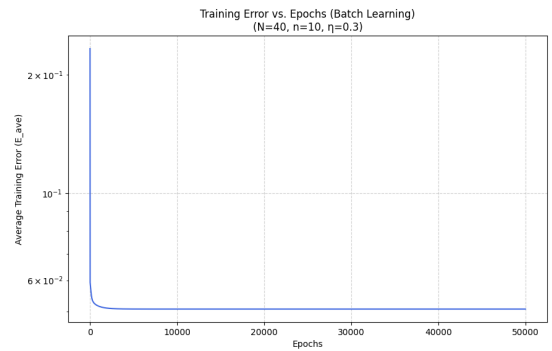


(b) Training Error vs. Epochs

Figure 50: Batch Learning Results for $N=40$, $n=10$, $\eta=0.1$.

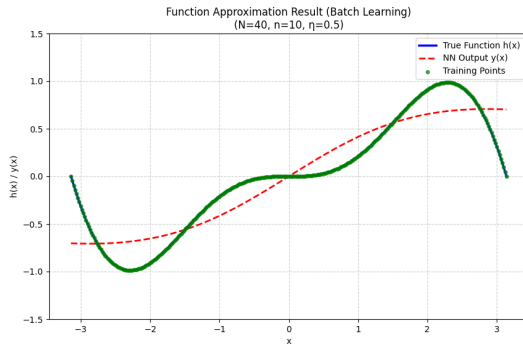


(a) Function Approximation Plot

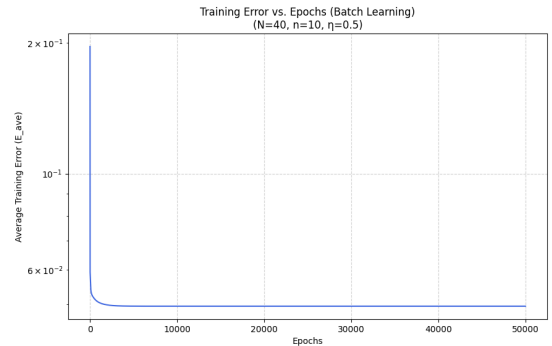


(b) Training Error vs. Epochs

Figure 51: Batch Learning Results for $N=40$, $n=10$, $\eta=0.3$.

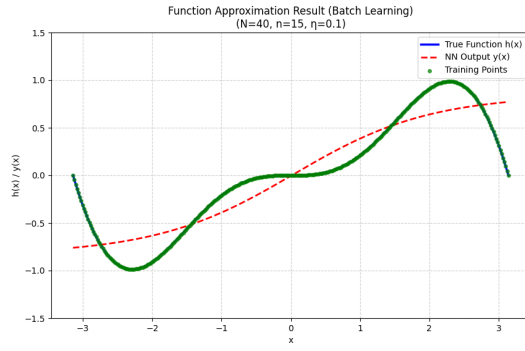


(a) Function Approximation Plot

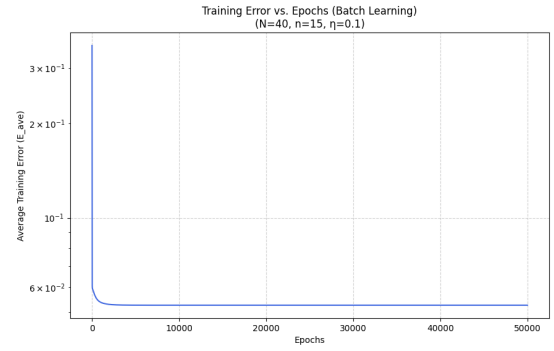


(b) Training Error vs. Epochs

Figure 52: Batch Learning Results for $N=40$, $n=10$, $\eta=0.5$.

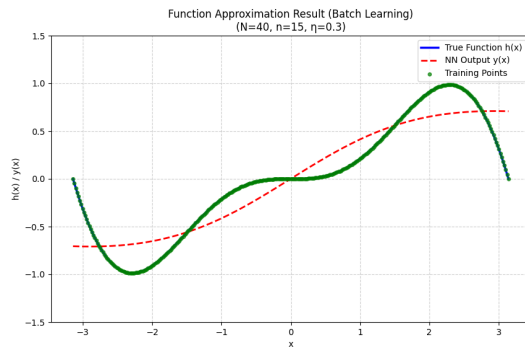


(a) Function Approximation Plot

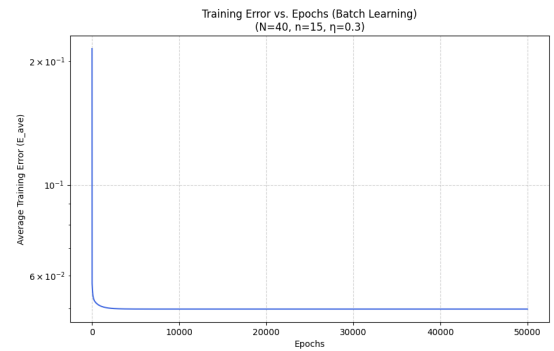


(b) Training Error vs. Epochs

Figure 53: Batch Learning Results for $N=40$, $n=15$, $\eta=0.1$.

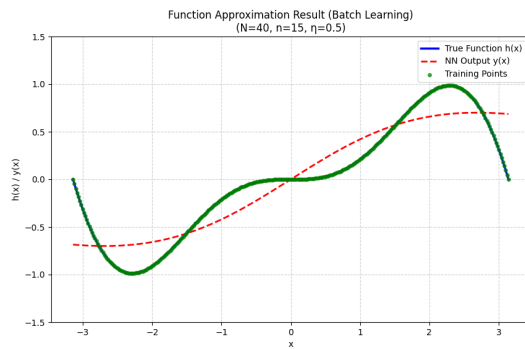


(a) Function Approximation Plot

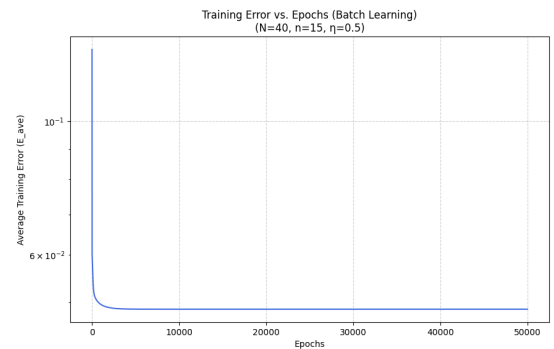


(b) Training Error vs. Epochs

Figure 54: Batch Learning Results for $N=40$, $n=15$, $\eta=0.3$.

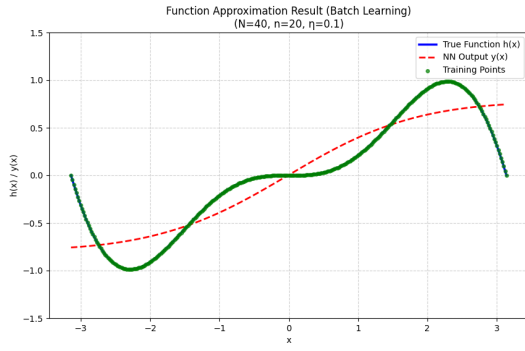


(a) Function Approximation Plot

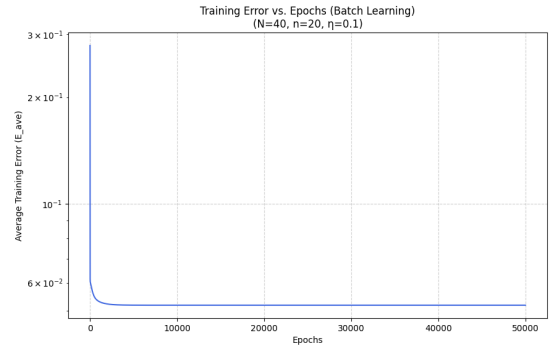


(b) Training Error vs. Epochs

Figure 55: Batch Learning Results for $N=40$, $n=15$, $\eta=0.5$.

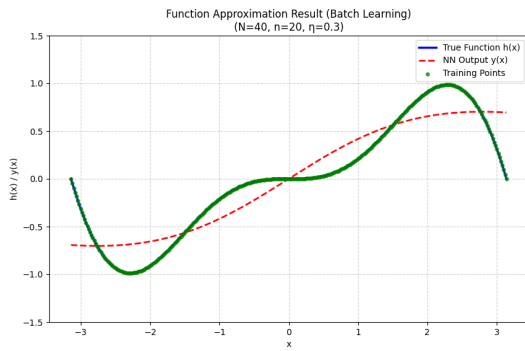


(a) Function Approximation Plot

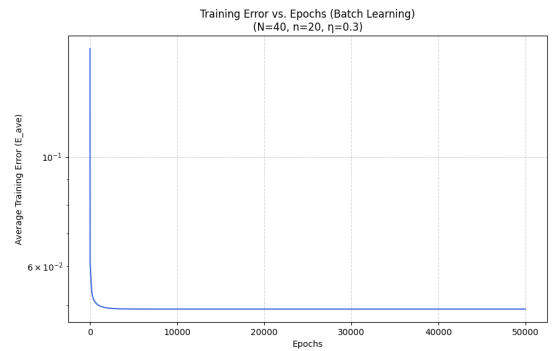


(b) Training Error vs. Epochs

Figure 56: Batch Learning Results for $N=40$, $n=20$, $\eta=0.1$.

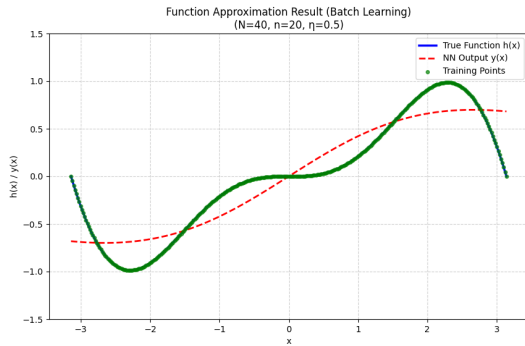


(a) Function Approximation Plot

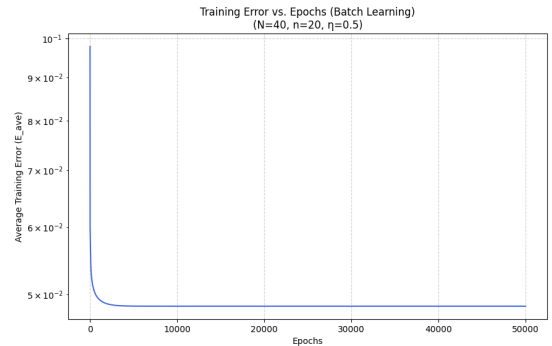


(b) Training Error vs. Epochs

Figure 57: Batch Learning Results for $N=40$, $n=20$, $\eta=0.3$.



(a) Function Approximation Plot



(b) Training Error vs. Epochs

Figure 58: Batch Learning Results for $N=40$, $n=20$, $\eta=0.5$.